

Tipo de datos compuestos

Juan Arnedo Moreno

Programación básica (ASX)
Programación (DAM)
Programación (DAW)

Índice

Introducción	5
Resultados de aprendizaje	7
1 Tipo de datos compuestos: "arrays"	9
1.1 Declaración e inicialización de arrays	10
1.1.1 Inicialización a un valor concreto	11
1.1.2 Inicialización a un valor por defecto	12
1.2 Manipulación de datos dentro de "arrays"	13
1.3 Almacenamiento de la entrada de datos en un "array"	15
1.3.1 Entrada de secuencias de valores por teclado	15
1.3.2 "Arrays" completamente ocupados	18
1.3.3 "Arrays" parcialmente ocupados	20
1.4 Esquemas fundamentales en la utilización de arrays	22
1.4.1 Inicialización procedural	22
1.4.2 Recorrido	24
1.4.3 Búsqueda	25
1.4.4 Uso de "arrays"	27
auxiliares 1.4.5 Copia :	29
1.4.6 Cambio de tamaño	30
1.4.7 Ordenación	32
1.5 Arrays multidimensionales	34
1.5.1 Declaración de arrays bidimensionales	35
1.5.2 Esquemas de uso de arrays bidimensionales	36
1.5.3 Arrays de más de dos dimensiones	40
1.6 Soluciones de los retos propuestos :	43
2 Tratamiento de cadenas de texto	47
2.1 La clase "String"	47
2.1.1 Inicialización de objetos "String"	48
2.1.2 Manipulación de objetos	48
2.1.3 Acceso a los caracteres de una cadena de texto	50
2.1.4 Concatenación de cadenas de texto	52
2.1.5 "Arrays" de cadenas de texto	52
2.2 Entrada de cadenas de texto	53
2.2.1 Lectura desde teclado	54
2.2.2 Argumento del método principal	59
2.3 Manipulación de cadenas de texto	60
2.3.1 Búsqueda	61
2.3.2 Comparación	63
2.3.3 Creación de subcadenas	64
2.3.4 Transformaciones entre cadenas de texto y tipos primitivos	67
2.4 Soluciones de los retos propuestos	70

Introducción

Cualquier dato manipulado dentro de un programa pertenece a alguno de los tipos primitivos, puesto que son los únicos que sabe procesar el hardware del ordenador.

Normalmente, estos datos son almacenados dentro de variables, uno de los elementos básicos de un programa de cara a manipular datos. Esta aproximación es útil siempre que el número de datos a tratar sea relativamente limitado.

Ahora bien, precisamente, uno de los aspectos en los que se hace especialmente recomendable la resolución de un problema usando un programa de ordenador es cuando el volumen de datos a procesar es muy grande. Suponga que en lugar de trabajar con varios valores independientes, que necesita guardar y procesar, debe trabajar con muchos: 50, 100, 2.000 o muchos más. En este caso, la aproximación de guardar cada valor en una variable dentro del programa haría la tarea del programador muy farragosa. ¡El código fuente tendría más instrucciones de declaración de variables que quizás para resolver el problema!

Para solucionar esta cuestión, los lenguajes de programación ofrecen unos tipos de datos especiales, llamados tipos compuestos, que permiten aglutinar en una única variable una cantidad arbitraria de datos individuales pertenecientes a algún tipo concreto. Éstas quedan empaquetadas de una manera fácil de gestionar en la hora de consultar y modificar cada valor individual.

En el apartado "Tipo de datos compuestos: arrays" se presenta el tipo compuesto de datos más básico, el array, disponible en la mayoría de lenguajes de alto nivel.

Éste permite almacenar en una única variable un conjunto de valores en forma de secuencia, con la única restricción de que todos pertenezcan al mismo tipo de datos.

De esta forma, en una única variable podemos disponer de cualquier número de enteros, reales, etc. En el apartado verá la sintaxis en Java y los mecanismos más habituales de tratamiento de datos almacenados en su interior.

Aparte de los arrays, existen otros tipos compuestos de datos de diferentes grados de complejidad y con el objetivo de solucionar problemáticas similares, pero con ciertos matices. Un ejemplo es la manipulación de texto. El tipo primitivo carácter, por sí mismo, no tiene excesiva utilidad, ya que normalmente las personas usamos palabras o, al menos, texto de longitud arbitraria para comunicarnos. Por tanto, gestionar este tipo de información carácter por carácter también es poco viable. Sería interesante disponer de algún mecanismo para gestionar de forma sencilla una combinación cualquiera de varios caracteres. Ésta es precisamente la tarea destinada al tipo de datos llamado cadena de texto.

En el siguiente apartado, "Tratamiento de cadenas de texto", se presenta con más detalle este tipo compuesto de datos muy útil, llamado String en el lenguaje Java.

Si bien ya se había presentado de forma superficial en apartados anteriores, verá cómo usarlo para realizar tareas más complejas que mostrar texto por pantalla, cómo procesar los caracteres que contiene o convertir el texto a otros tipos primitivos.

Resultados de aprendizaje

Al finalizar esta unidad, el alumno/a:

1. Escribe y prueba programas sencillos reconociendo y aplicando los fundamentos de la programación modular.

1. Tipo de datos compuestos: "arrays"

Suponga que desea realizar un programa para analizar las notas finales de un conjunto de estudiantes. Las operaciones a realizar pueden ser diversas: calcular la nota media, ver cuántos estudiantes han aprobado, suspendido o han superado cierto umbral de nota, realizar gráficas de rendimiento, etc. Además, no se descarta que en el futuro el programa se modifique para añadir nuevas funcionalidades. El quid de la cuestión es que, para poder prever todas estas posibilidades y otras futuras todavía desconocidas, necesita disponer dentro del programa del valor de cada nota individual. En un caso como éste, tal y como se ha planteado hasta ahora la aproximación para manipular datos dentro de un programa, esto implica que cada nota debe almacenarse dentro de una variable diferente. Si hay 50 estudiantes, esto significa que habría que declarar 50 variables.

Desgraciadamente, esta aproximación no funciona en un caso como éste, en el que el número de datos a procesar es relativamente alto. Por un lado, imagínense el aspecto que tendrá un código fuente en el que hay que declarar y manipular 50 variables para realizar todo tipo de cálculos. Las expresiones para realizar cálculos entre ellas, como simplemente hacer la media (sumarlas todas y dividir por el número de valores), serían enormes. Por otra parte, ¿qué ocurre si es necesario procesar las notas de doscientos o mil estudiantes? ¿Es factible declarar mil variables? Peor aún: ¿qué pasa si el número de estudiantes de cada curso es totalmente diferente? En cada curso el número de variables usadas no encajará con el de estudiantes y será necesario modificar el código fuente teniendo en cuenta el número exacto de valores para tratar y volver a compilarse.

Ya a simple vista se puede apreciar que todo esto es inviable. Es necesario un sistema flexible para almacenar un número arbitrario de valores de forma que éstos sean fáciles de manipular. Es más, debe ser posible que el número de valores almacenados pueda ser diferente para cada ejecución del programa. Para resolver esta problemática, los lenguajes de programación de alto nivel ofrecen el tipo de dato compuesto array, o tabla.

Un tipo de dato compuesto es aquél que permite almacenar más de un valor dentro de una única variable. En el caso del array, éste permite almacenar, en forma de secuencia, una cantidad predeterminada de valores pertenecientes al mismo tipo de datos.

Para diferenciar los diferentes valores almacenados, el array gestiona su contenido de acuerdo con posiciones que siguen un orden numérico: el valor almacenado en la primera posición, en la segunda, en la tercera, etc. A efectos prácticos, puede considerarse que cada posición individual se comporta exactamente igual que una variable del tipo de dato elegido. Tanto se puede consultar el valor como almacenar datos. Este comportamiento se esquematiza en la figura 1.1.



Un "array" es como una cómoda.
Imagen de Liverpool Design Festival

Figura 1. 1. Organización de los datos almacenados dentro de un array



Antes de ver con más detalle cómo funcionan, debe tener presente que la sintaxis para usar arrays en los distintos lenguajes de programación puede ser relativamente diferente. Este apartado se centrará en la sintaxis concreta de Java, que tiene ciertas particularidades únicas. Sin embargo, el concepto general de acceso a valores ordenados como una secuencia en forma de array es aplicable a cualquier lenguaje.

1.1 Declaración e inicialización de "arrays"

Como ocurre con datos de cualquier otro tipo, antes de poder hacer uso de una variable de tipo array, es necesario declararla y otorgarle un valor inicial correctamente. La sintaxis para declarar un array es bastante similar a la de otros tipos, pero la inicialización tiene unas particularidades especiales debido a sus propias características:

- Por un lado, es necesario decidir de cuántas posiciones dispondrá el array: su tamaño. Nunca podrá haber almacenados más valores que su capacidad.
- Por otro lado, decidir el valor inicial para cada posición individual del array. En este aspecto, cada posición tiene el mismo papel que una variable individual dentro del programa, y por tanto, debe estar inicializada con algún valor.

En el momento de la inicialización de un array, su tamaño debe ser conocido. Una vez inicializado con un número de posiciones concretas, ésta ya no puede cambiar.

Antes de poder utilizar un array es obligatorio inicializarlo. Java dará un error de compilación de lo contrario.

1.1.1 Inicialización a un valor concreto

Como ocurre con las variables individuales, existe la posibilidad de que en el momento de generar el código fuente ya tenga unos valores concretos en la cabeza con los que desea inicializar cada una de las posiciones del array. En este caso, la sintaxis de la inicialización es la siguiente:

```
1 palabraClave[] identificadorVariable = {valor1, valor2, ..., valorN};
```

La parte izquierda es prácticamente igual que para la declaración de cualquier variable. Lo que indica que en realidad se está declarando un array son los dos corchetes [] inmediatamente después de la declaración del tipo. Cada una de las posiciones del array sólo podrá almacenar valores del tipo aquí declarado.

No puede haber datos de tipos mezclados dentro de un array.

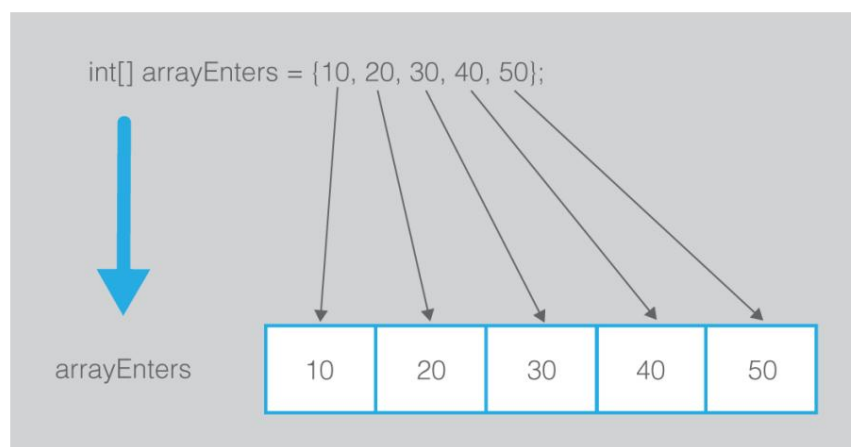
Para especificar los valores almacenados inicialmente en cada posición es suficiente que estos se enumeren entre claves {...}, separados por comas (,). Estos valores serán del mismo tipo de dato que el declarado en la parte izquierda y se almacenarán en el mismo orden en que se hayan enumerado: el primer valor en la primera posición del array, el segundo en la segunda posición, etc. En este caso, el tamaño del array será automáticamente al igual que el número de valores enumerados.

Por ejemplo, para declarar un array de 5 posiciones en el que se pueden almacenar valores de tipo entero, cada una con los valores iniciales 10, 20, 30, 40 y 50, respectivamente, se haría de la siguiente forma. El tamaño de este array sería 5.

```
1 int[] arrayEnters = {10, 20, 30, 40, 50};
```

El resultado se muestra en la figura 1.2.

Figura 1. 2. Esquema de inicialización de un array en Java con valores concretos.



1.1.2 Inicialización a un valor por defecto

Muchas veces los datos que se quieren almacenar dependen de la lectura de una entrada (por ejemplo, por teclado), o son resultado de cálculos que son más cómodos de llevar a cabo de forma automatizada dentro del código de programa (por ejemplo, una lista de 1.000 números primos). En este caso, no existe un valor inicial concreto que sea necesario asignar a las posiciones del array, ya que los valores se almacenarán a posteriori.

En este caso, otra forma, más simple y habitual, de declarar e inicializarlo, es asumiendo que todas las posiciones toman automáticamente un valor por defecto, que es 0 para los datos de tipo numérico y los caracteres, y false para booleanos.

Más adelante ya asignará a cada posición el valor que desee.

Esto se hace de la siguiente forma:

```
1 palabraClauTipus[] identificadorVariable = new palabraClauTipus[tamaño];
```

Sólo varía la parte derecha relativa a la asignación de los valores iniciales. En este caso la sintaxis es muy estricta y es necesario utilizar el formato indicado. Se usa la palabra clave `new`, seguida nuevamente del tipo de los datos dentro del array `y`, entre corchetes, el tamaño que queramos. Los tipos de datos especificados en la parte izquierda y derecha deben ser idénticos, o habrá un error de compilación.

No es necesario hacer nada más para asignar valores a las posiciones del array, Java ya asigna automáticamente el valor por defecto. Ahora bien, alerta, ésta es una propiedad específica de Java y no debe ser así necesariamente en otros lenguajes, en los que el valor inicial de cada posición puede ser indeterminado.

Por ejemplo, para declarar un array de 10 posiciones en el que se pueden almacenar valores de tipo real, todos inicialmente con el valor 0, se haría de la siguiente forma:

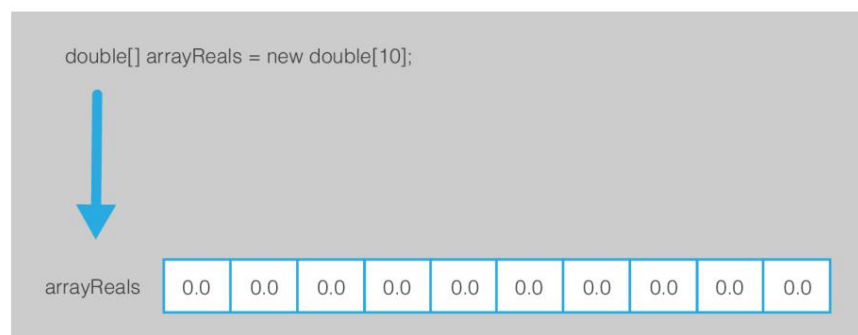
```
1 double[] arrayReals = new double[10];
```

Este código es equivalente a:

```
1 double[] arrayReals = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
```

El resultado de realizar cualquiera de las dos instrucciones se visualiza en la figura 1.3.

Figura 1.3. Esquema de inicialización de un array en Java con valores por defecto



1.2 Manipulación de datos dentro de "arrays"

A pesar de conformar un tipo de dato (como enteros, reales, etc.), los arrays son algo especiales a la hora de ser manipulados, ya que no disponen de ninguna operación. No es posible usar el identificador del array directamente para invocar operaciones y así manipular los datos contenidos, tal y como se puede realizar con variables de otro tipo. Por ejemplo, no es posible hacer lo siguiente:

En Java, realizar operaciones entre arrays comporta un error de compilación.

```
1 ...  
2 int[] a = {10, 20, 30, 40, 50}; 3 int[] b = {50, 60, 70, 80, 100}; 4 int[]  
c = a + b;  
5 ...
```

Siempre que se quiera realizar alguna operación con los datos almacenados dentro de arrays es necesario manipularlos de manera individual, posición por posición. En este aspecto, cada posición de un array tiene exactamente el mismo comportamiento que una variable tal y como las ha estudiado hasta ahora. Puede aprovecharlas tanto para almacenar datos como para leer su contenido.

Para distinguir entre las diferentes posiciones de un array, cada una tiene asignado un índice, un valor entero que indica su orden dentro de la estructura. Siempre que se quiera referir a una de las posiciones, basta con usar el identificador del array junto con el índice de la posición buscada entre corchetes []. La sintaxis exacta es:

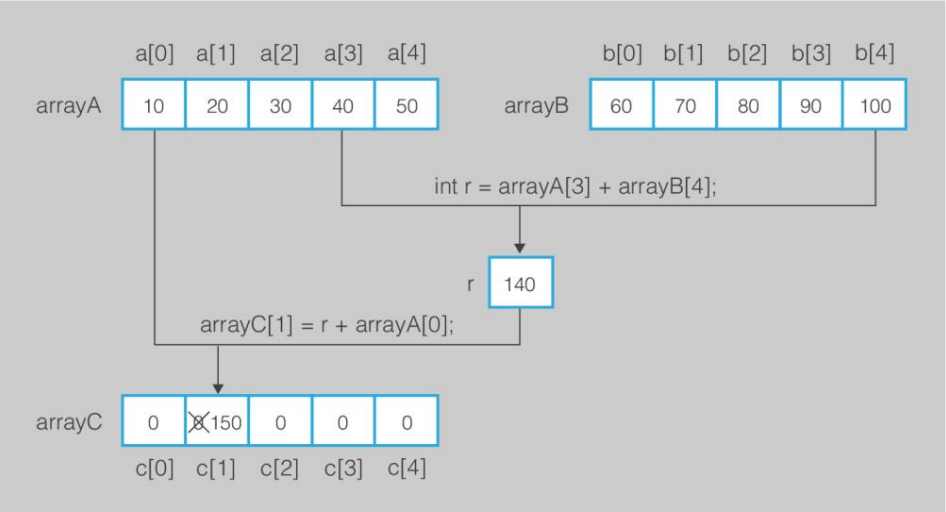
```
1 identificadorArray[indice]
```

El rango de los índices puede variar en función del lenguaje de programación. En el caso de Java, éstos van de 0, para la primera posición, a (tamaño - 1), para la última. Por ejemplo, para acceder a las posiciones de un array de cinco posiciones usaría los índices 0, 1, 2, 3 y 4. A continuación se muestra un ejemplo de cómo se accede a los datos de un array mediante los índices de sus posiciones:

```
1 ...  
2 int[] arrayA = {10, 20, 30, 40, 50}; 3 int[] arrayB = {50, 60, 70, 80, 100}; 4 int[]  
arrayC = new int[5]; 5 //La variable de tipo entero "r" valdrá 140, 40 + 100. 6 int  
r = arrayA[3] + arrayB[4]; 7 //La segunda posición (índice  
1) de "arrayC" valdrá 150, 140 + 10. 8 arrayC[1] = r + arrayA[0];  
9 ...
```

La figura 1.4 esquematiza el acceso a los datos dentro de ese código.

F igura 1. 4. Ejemplo de manipulación de datos dentro de "arrays"



Es muy importante que al acceder a un array se utilice un índice correcto, en función del rango admisible. Si se utiliza un valor no admisible, como por ejemplo un valor negativo o un valor igual o superior a su tamaño, habrá un error de ejecución.

Concretamente, se producirá una `IndexOutOfBoundsException`. Si esto sucede, puede tener la certeza de que el programa tiene un error y deberá repasarlo y corregirlo.

Por tanto, el siguiente código sería incorrecto:

```
1 ...
2 //arrayA" tiene tamaño 5.
3 //Los índices válidos van de 0 a (tamaño - 1), que es 4. 4 int[] arrayA = {10,
20, 30, 40, 50}; 5 //Se asigna un valor a una posición
incorrecta (índice 5). 6 arrayA[5] = 60; 7 //El índice 6 es inválido, no se puede
consultar el valor. 8 int r
= arrayA[2] + arrayA[4] + arrayA[6];
9 ...
```

Con el fin de ayudarle en la tarea de controlar si un índice concreto está dentro del rango admisible para una variable de tipo array concreta, Java dispone de una herramienta auxiliar, el atributo `length` (largo). Su sintaxis es la siguiente:

```
1 identificadorArray.length
```

El resultado de esta instrucción es equivalente a evaluar una expresión que como resultado obtiene el tamaño del array llamado `identificador Array`. Ahora bien, recuerde que `length` le da el tamaño, pero el rango de índices válido es `0 ... (tamaño - 1)`. Por ejemplo:

```
1 ...
2 //arrayA" tiene tamaño 5. Los índices válidos van de 0 a 4. 3 int[] arrayA = {10, 20, 30, 40, 50}; 4 //length" le
dice el tamaño.

5 //Recuerde que el índice máximo es (tamaño - 1). 6 if (index <
arrayA.length) {
7     System.out.println("La posición " + index + " vale " + a[index]);

8 }
9 System.out.println("Este array no tiene tantas posiciones!");
10 }
11 ...
```

1.3 Almacenamiento de la entrada de datos en un "array"

Como se ha mencionado, los arrays son especialmente útiles para poder almacenar de forma eficiente un número arbitrario de datos provenientes del sistema de una entrada (por ejemplo, desde el teclado). Normalmente, los datos se irán leyendo una por una, ya medida que se haga, habrá que ir asignando a cada una de las posiciones del array a partir de su índice. A continuación verá diferentes esquemas para realizar esta tarea.

Si bien los ejemplos se centrarán en el uso del teclado, ya que es el sistema de entrada que por ahora sabe utilizar, debe tener en cuenta que estos esquemas serán aplicables a cualquier otro mecanismo de entrada (por ejemplo, un archivo). La única condición es que los datos estén en forma de secuencia, de modo que se puedan ir leyendo una por una, ordenadamente.

Una vez expuestos estos esquemas, en el resto de ejemplos del apartado se obviará el código relativo a la entrada de datos por el teclado.



Almacenar en "arrays" datos leídos secuencialmente es como una cadena de embotellamiento

1.3.1 Entrada de secuencias de valores por teclado

Los arrays tienen sentido cuando el programa debe procesar muchos datos, especialmente provenientes de la entrada del programa (como el teclado). Antes de continuar, vale la pena ver cómo se pueden leer secuencias de datos entrados desde el teclado en una sola línea de texto, de modo que sea sencillo almacenarlos dentro de un array. Si bien ya sabe cómo leer datos individualmente desde el teclado, hacerlo así puede ser un poco aburrido y molesto de cara a la ejecución de los programas en los que se introducen muchos datos, preguntando cada valor uno por uno y debiendo pulsar la tecla de retorno cada vez.

En realidad, cuando se usa una instrucción `lector.next...` y el programa se detiene esperando a que el usuario introduzca datos por el teclado, nada impide que, en lugar de un único dato, éste escriba más de uno antes de pulsar la tecla de retorno.

O sea, una secuencia de valores. La única condición es que cada valor individual esté separado del resto por al menos un espacio, de modo que sean fácilmente identificables. Cuando esto sucede, todos los valores de la secuencia quedan latentes, pendientes de lectura. Sucesivas invocaciones a nuevas instrucciones de lectura automáticamente irán consumiendo estos valores pendientes, en lugar de causar una nueva espera de una entrada del usuario. Mientras queden valores en la secuencia pendientes de leer, las instrucciones de lectura nunca causarán una espera de entrada de datos. Una vez que la secuencia ha sido consumida totalmente, entonces sí que la próxima instrucción de lectura causará que el programa vuelva a detenerse esperando una nueva entrada del usuario. Y así el ciclo de lectura vuelve a empezar.

Dentro de este proceso existe una llamada con un comportamiento especial: `lector.nextLine()`. Cuando esta llamada se invoca, automáticamente se descarga

tienen todos los valores pendientes de leer de la secuencia actual. La próxima llamada a una instrucción de lectura siempre causará que el programa se detenga de nuevo y espere una entrada del usuario.

La tabla 1.1 muestra un ejemplo de este mecanismo de lectura, en el que se leen valores de tipo entero.

Tabla 1. 1. Ejemplo de lectura de una secuencia de valores enteros

Acción del programa	Resultado obtenido	Datos pendientes de lectura
(Inicio del programa)	El programa se inicia	{ } (no hay ninguna)
Leer valor	El programa espera entrada de el usuario	{ } (no hay ninguna)
	El usuario entra los valores "1 2 4 8 16 32" y pulsa la tecla de retorno	
	La lectura evalúa 1	{ 2, 4, 8, 16, 32 }
Leer valor	La lectura evalúa 2	{ 4, 8, 16, 32 }
Leer valor	La lectura evalúa 4	{ 8, 16, 32 }
Leer valor	Se vacía la secuencia de datos pendientes	{ }
Leer valor	El programa espera entrada de el usuario	{ }
Se vuelve a empezar el ciclo de lectura de una nueva secuencia de valores...		

Un hecho importante cuando se leen secuencias de varios valores escritas desde el teclado es que nada impide que pueda haber valores de diferentes tipos de datos mezclados. Esto tanto puede ser por error del usuario al entrar los datos, como para que programa realmente espera una secuencia con valores de distintos tipos intercalados. En cualquier caso, es importante que antes de realizar ninguna lectura de un dato se compruebe si el tipo es el que corresponde usando las instrucciones `lector.hasNext...`

Para leer múltiples valores lo más sencillo es hacerlo mediante una estructura de repetición que vaya invocando sucesivamente las instrucciones de lectura de datos. Al usar este mecanismo es necesario tener presente un hecho muy importante. Hay que saber exactamente cuando ha obtenido ya todos los datos necesarios y es necesario dejar leer. Normalmente hay dos aproximaciones dependiendo de si se conoce la cantidad de datos exactos que deben leerse o no.

Cantidad de datos conocida

Si el número de valores que se desea leer es conocido de antemano, para tratar la lectura de la secuencia de valores basta con usar una estructura de repetición basada en un contador. Este contador controlará el número de valores leídos y las iteraciones finalizarán al alcanzar el número de lecturas que queramos.

Compile y ejecute el siguiente ejemplo, en el que se muestra cómo hacer la lectura de una serie de valores enteros. Si se escriben más de los esperados, el resto se descartan. Tenga en cuenta que no es necesario escribir todos los valores en una sola línea de texto. Si la línea introducida, antes de pulsar la tecla de retorno, no dispone de suficientes valores, el programa se detiene esperando el resto. Además, este código

controla mediante una estructura de selección si el valor que está a punto de ser leído será realmente un entero o no.

```
1 importe java.util.Scanner;
2 //Un programa que lee un número conocido de valores enteros.
3 public class LectorValoresConocidos {
4     //Se leerán 10 valores.
5     public static final int NUM_VALORS = 10;
6     public static void main (String[] args) {
7         Scanner lector = new Scanner(System.in);
8         System.out.println("Escribe " + NUM_VALORS + " enteros. Se puede hacer en
          líneas.");
9         //Se leen exactamente NUM_VALORS valores.
10        int numValorsLeídos = 0;
11        while (numValorsLeídos < NUM_VALORS) {
12            //Antes de leer, comprobamos si realmente existe un entero.
13            if (lector.hasNextInt()) {
14                int valor = lector.nextInt();
15                System.out.println("Valor" + numValorsLeídos + numValorsLeídos+ " leído: " + valor);
16                +;
17            } else {
18                //Si el valor no es entero, se lee pero se ignora.
19                //No se adelanta tampoco el contador.
20                lector.next();
21            }
22        }
23        //Los valores que se sobren en la última línea escrita se descartan.
24        lector.nextLine();
25        System.out.println("Ya se han leído " + NUM_VALORS + " valores.");
26    }
27 }
```

Cantidad de datos desconocida

Un caso más complejo es cuando el número de valores no es conocido a priori, ya que puede variar en distintas ejecuciones del programa. Cuando esto sucede, una solución simple sería preguntar simplemente, antes de leer ningún valor, ¿cuántos datos se introducirán, o bien hacer que el primer valor dentro de la secuencia indique la longitud.

Si esto no es posible, o se considera que no debe hacerse, otra opción es establecer un valor especial que no forma parte de los datos a tratar dentro del programa, sino que se considerará como marca de fin de secuencia. En cuanto se lea este valor, la lectura debe darse por finalizada. La condición para poder usar esta estrategia es que la marca no sea un valor que pueda aparecer nunca entre los valores introducidos. Debe ser único. Por ejemplo, en una lista de notas, cualquier valor negativo o mayor que 10 serviría. El resto de valores no se pueden usar, ya que pueden corresponder a una nota real a tratar.

Si todos los valores posibles son aceptables dentro de la secuencia, una opción es usar un valor de un tipo de dato distinto. Por ejemplo, un programa que procesa valores reales arbitrarios para realizar cálculos matemáticos podría usar un carácter como marca de fin.

En este caso, la estrategia a utilizar es usar una estructura de repetición basada en semáforo. Éste le indica si ya se ha leído la marca de fin y hay que dejar de iterar o no.

El siguiente código de ejemplo lee una secuencia de valores enteros de longitud arbitraria. La lectura finaliza en cuanto se lee el valor -1. Compare este código con el ejemplo del caso anterior.

```
1 importe java.util.Scanner;
2 public class LectorValoresDeconocidos {
3     public static final int MARCA_FI = -1;
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         System.out.print("Escribe diferentes valores enteros.");
7         System.out.println("Después del último valor escribe un //Se leen exactamente " + MARCA_FI);
8         NUM_VALORS valores.
9         boolean marcaEncuentro = false;
10        while (!marcaEncuentro) {
11            //Antes de leer, comprobamos si realmente existe un entero.
12            if (lector.hasNextInt()) {
13                int valor = lector.nextInt();
14                //¿Es la marca de fin?
15                if (valor == MARCA_FI) {
16                    //Sí que lo es.
17                    marcaEncuentro = true;
18                } else {
19                    //No. Es un valor que debe ser tratado.
20                    System.out.println("Valor leído: " + valor);
21                }
22            } else {
23                //Si el valor no es entero, se lee pero se ignora.
24                lector.next();
25            }
26        }
27        //Los valores que se sobren en la última línea escrita se descartan.
28        lector.nextLine();
29        System.out.println("Ya se han leído todos los valores.");
30    }
31 }
```

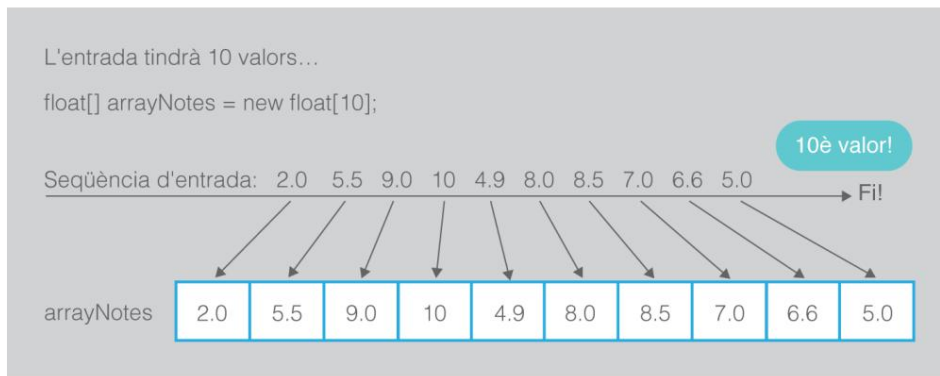
1.3.2 "Arrays" completamente ocupados

El esquema más simple de almacenamiento de datos desde una entrada a las posiciones de un array es el caso en que la cantidad de datos que se leerá se conoce previamente. Esto se debe a la importante restricción que, para inicializar un array, previamente se debe establecer su tamaño. Éste es un requisito indispensable, ya que en caso contrario cualquier intento de guardar datos resultará en un error de ejecución. En este caso, basta con inicializar el array con sus posiciones con valor por defecto y un tamaño igual al número de datos que se leerán. Una vez hecho, basta con leer de la entrada un número de datos esperado e ir asignando los valores de estos datos en cada una de las posiciones del array, consecutivamente.

Un hecho que debe tener cuidado es que puede que se indique que el número de datos para entrar es un valor, pero después, por error, se proporcionen más. En cualquier caso, nunca se pueden leer más datos que la capacidad real del array, por lo que todos estos datos sobrantes se ignorarán. Una vez el array está en el máximo de su capacidad, se debe terminar el proceso de lectura o se exponga a un error de ejecución al sobrepasar el tamaño del array.

La figura 1.5 muestra un esquema de la lectura de una secuencia de valores a ocupar totalmente un array.

Figura 1. 5. Esquema de entrada de datos para crear un "array" totalmente ocupado



Como ejemplo, suponga un programa para tratar las notas de evaluación de un conjunto de estudiantes. Para obtener las diferentes notas, éste lee una secuencia, de longitud conocida, compuesta de números reales. Con los valores leídos se llena completamente un array. Compile y ejecute el siguiente código para llevar a cabo esta operación. Fíjese que, en este caso, se comprueban una serie de errores posibles en la entrada de los datos antes de almacenar el valor final en el array: si los valores introducidos son realmente de tipo real y si están dentro del rango válido por a una nota (entre 0 y 10).

```

1 importe java.util.Scanner;
2 //Programa que lee una secuencia de valores reales, de longitud conocida.
3 public class LectorSecuencia {
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         //Lee la longitud de la secuencia. Comprueba todos los errores.
7         int númeroValores = 0;
8         while (númeroValores <= 0) {
9             System.out.print("¿Cuántas notas quieres introducir? ");
10            if (lector.hasNextInt()) {
11                númeroValores = lector.nextInt();
12            } else {
13                //Si no es entero, se lee y se ignora.
14                lector.next();
15            }
16        }
17        //Si se han entrado más valores, se ignoran. Sólo se necesita uno.
18        lector.nextLine();
19        System.out.println("Se leerán " + númeroValores + " valores reales.");
20        System.out.println("Puedes escribir varios en una sola línea, separados
    por espacios.");
21        //Los guardaremos en un array. Ya conocemos su tamaño.
22        float[] arrayNotes = new float[númeroValores];
23        //Hay que leer tantos reales como el tamaño del array.
24        //Estructura de repetición con contador.
25        int índice = 0;
26        while (index < arrayNotes.length) {
27            if (lector.hasNextFloat()) {
28                //Se ha leído una nota, pero ¿es válida (entre 0 y 10)?
29                float nota = lector.nextFloat();
30                if ((nota >= 0)&&(nota <= 10)) {
31                    arrayNotes[index] = nota;
32                    index++;
33                }
34                //Si no es válida, la ignoramos. No se asigna a ninguna parte.
35            } else {
36                //Si no era un real, simplemente leemos el valor como una cadena de texto
37                //pero no hacemos nada. Se pierde.
38                lector.next();
39            }

```

```

40     }
41     //Ignoramos los valores sobrantes de la última línea. lector.nextLine();
42     System.out.println("La
43     secuencia leída es:"); for (int i = 0; y < arrayNotes.length;i++)
44     { System.out.println(arrayNotes[i]);
45
46     }
47 }
48 }

```

1.3.3 "Arrays" parcialmente ocupados

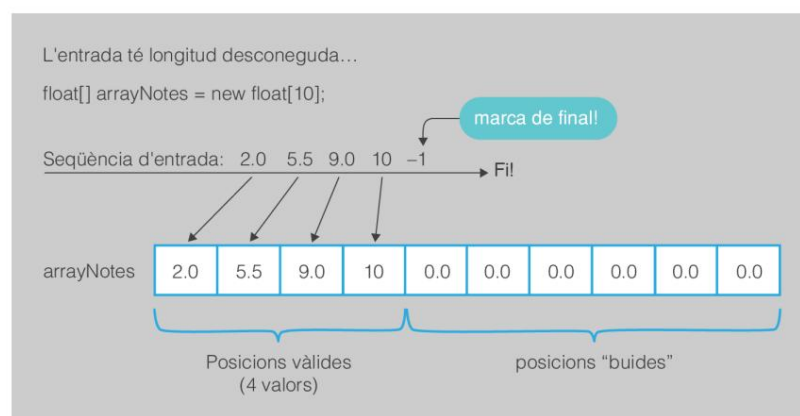
Desgraciadamente, hay circunstancias bajo las cuales no es posible inicializar un array de forma que su tamaño se ajuste exactamente a la longitud de la secuencia de valores de entrada. Hay dos motivos que pueden llevar a esa situación. Por un lado, si simplemente no se conoce a priori el número de datos que se van a entrar, ya que en lugar del programa se pregunta previamente, y por tanto la entrada de datos es de una longitud desconocida. Por otra parte, esto también sucede en lenguajes de programación en los que a la hora de declarar un array e inicializarlo no es posible utilizar una variable para especificar su tamaño, y sólo se puede usar un literal.

Cada posición de un array ocupa tanta memoria como el tipo primitivo que puede almacenar.

En un caso como éste, deberá conformarse con elegir un valor concreto prefijado para el tamaño del array y dar por sentado que el número de datos de entrada que el programa puede tratar es limitado. Si por algún motivo, al leer los datos de entrada, se supera este límite, al no ser posible almacenar los valores excedentes, deberá ignorarlos. Por tanto, habrá que elegir este tamaño llegando a un compromiso entre un valor que sea lo suficientemente grande para no encontraros muy a menudo con esta situación, pero tampoco demasiado exagerado, para no derrochar memoria. Por ejemplo, si ya sabe que el número de estudiantes en una clase puede oscilar, pero nunca se permitirá que haya más de 80 (en este caso, se abriría una nueva aula), se puede elegir este valor como tamaño del array. Es necesario estudiar cada caso concreto.

Coloquialmente, una posición en la que no existe ningún valor válido asignado se dice que está "vacía".

Figura 1. 6. Esquema de entrada de datos que genera un "array" parcialmente ocupado



En un caso como éste, en el que se dispone de un array con un tamaño que nunca será exactamente igual al número de datos leídos desde la entrada (de hecho, normalmente, será inferior), sólo existe un rango de posiciones en el que hay datos leídos de

la entrada. El resto de posiciones tienen valores por defecto que nada tienen que ver con la entrada, y por tanto, no deben ser previstas a la hora de realizar ninguna tarea. La figura 1.6 esquematiza este hecho partiendo de la secuencia de entrada 2.0, 5.5, 9.0, 10.

Otro aspecto importante es que el atributo `length` ahora no es suficiente para saber el rango de los datos válidos. Sólo indica la capacidad del array. Por tanto, será necesario disponer de una variable auxiliar adicional, de tipo entero, en la que se almacene el número de datos que realmente hay dentro del array. Cualquiera acceso al array sólo obtendrá datos correctos si el índice empleado se encuentra entre 0 y este valor auxiliar - 1.

El siguiente código es el equivalente de entrada de notas pero para el caso de una entrada de datos de longitud desconocida. Como no se sabe cuántos datos hay que leer, es necesario indicar de alguna manera cuando se ha terminado de entrar datos, con una marca de final. En ese caso, se trata del valor -1. El límite de datos que es capaz de almacenar se ha establecido en 80, definido como la constante `MAX_VALORS`.

Pruébalo en su entorno de trabajo y estudie su funcionamiento.

```
1 importe java.util.Scanner;
2 //Lee un conjunto de datos sin saber cuántos son.
3 public class LectorSecuenciaDimDesconocida {
4     public static final int MAX_VALORS = 80;
5     public static final int MARCA_FI = -1;
6     public static void main (String[] args) {
7         //No se sabe cuántas entrarán. Hay que acotar un valor máximo. Ponemos 80.
8         float[] arrayNotes = new float[MAX_VALORS];
9         //Es necesario un contador de posiciones actuales en las que hay valores.
10        int elementos = 0;
11        Scanner lector = new Scanner(System.in);
12        System.out.print("Escribir hasta " + MAX_VALORS + " valores.");
13        System.out.println("Puedes escribir varios en una sola línea, separados
14        por espacios.");
15        //Será necesario alguna manera de saber que se ha terminado de escribir.
16        System.out.println("Para terminar, escribe un " + MARCA_FI + " ");
17        //Ir leyendo hasta encontrar la marca de fin. Pero el máximo es el tamaño del
18        array.
19        //Si se entran más valores, los ignoramos, ya que no queda lugar donde almacenar
20        -los.
21        //Estructura de repetición con semáforo.
22        while (elementos < arrayNotes.length) {
23            //¿Hay un real?
24            if (lector.hasNextFloat()) {
25                //Hay que ver si es un valor válido o final de secuencia.
26                float nota = lector.nextFloat();
27                if ((nota >= 0)&&(nota <= 10)) {
28                    //Todo correcto. Ahora hay un elemento más en el array.
29                    arrayNotes[elementos] = nota;
30                    elementos++;
31                } else if (nota == MARCA_FI) {
32                    //Marca de final, salimos del bucle.
33                    break;
34                }
35            }
36            //Si no es ni lo uno ni lo otro, lo ignoramos.
37        } else {
38            //Si no era un real, lo ignoramos.
39            lector.next();
40        }
41    }
42    //Ignoramos los valores sobrantes de la última línea.
43    lector.nextLine();
44    System.out.println("La secuencia leída es:");
```

```
41     for (int i = 0; y < elementos;i++)  
42         { System.out.println(arrayNotes[i]);  
43     }  
44 }  
45 }
```

A un array parcialmente ocupado se le pueden añadir nuevos datos posteriormente, siempre que no se supere su tamaño. Para ello, éstas deberían asignarse a partir de la posición en la que se encuentra el último elemento válido.

1.4 Esquemas fundamentales en la utilización de "arrays"

El potencial de los arrays va mucho más allá de la mera capacidad de gestionar conjuntos de datos de tamaño arbitrario tan sólo declarando una única variable.

Éstos también son una herramienta extremadamente versátil para procesar todos los datos que contienen con pocas líneas de código de forma comprensible y eficiente. El principal motivo es que sus posiciones son accesibles a partir de índices numéricos.

Esto hace ideal el tratamiento usando estructuras de repetición, que mediante un contador accedan una por una a todas las posiciones.

De hecho, esta circunstancia ha quedado ya patente en el ejemplo de lectura de una secuencia de datos desde el teclado y el almacenamiento en un array. Reflexione sobre cómo sería el código sin usar una estructura de repetición o arrays.

A continuación verá una serie de estrategias basadas en esta circunstancia que vale la pena que tenga presentes a la hora de trabajar con los datos contenidos dentro de un array. Casi todos los programas basados en arrays usan alguna. Para simplificar el código de los ejemplos utilizados, en lugar de partir de una entrada por teclado se trabajará sobre un array inicializado con valores concretos. Los algoritmos no varían en absoluto respecto a los arrays producto de una entrada por teclado.

1.4.1 Inicialización procedural

Hay situaciones en las que los valores iniciales dentro de alguna o todas las posiciones del array no necesariamente deben depender de una entrada, pero usar el sistema de inicialización con valores concretos puede ser algo incómodo. Por ejemplo, suponga que necesita disponer de los valores de las 20 primeras potencias de 2. Al tratarse de bastantes valores, usar un array es ideal. Ahora bien, aunque estos valores se pueden expresar directamente como literales, primero debería calcularlos por su cuenta y, a continuación, inicializar el array. Además, el resultado final es una línea bastante larga dentro del código, afectando a su legibilidad.

Puestos a tener que calcular previamente por su parte los valores con los que se debe inicializar el array, sale más a cuenta que sea el propio programa quien realice este cálculo e inicialice las posiciones a medida que encuentre los resultados.

Se llama generación procedural al hecho de generar contenido algorímicamente, en lugar de hacerlo manualmente partiendo de valores preestablecidos. Es un término especialmente usado en el diseño de videojuegos.

Debido a que cuando se realiza una inicialización de este tipo el número de valores es conocido, se puede asignar un tamaño al array de forma que esté totalmente ocupado. Compile y ejecute el siguiente código, que inicializa proceduralmente un array de modo que el valor en cada posición sea el doble de la anterior, partiendo de una primera posición con valor 1.

```
1 //Un programa que inicializa proceduralmente un array con valores relacionados
  entre sí.
2 public class InicializaciónProcedural {
3     public static void main (String[] args) {
4         //Habr  que almacenar 20 enteros.
5         int[] arrayValorsDobles = new int[20];
6         //La primera posici n la ponemos directamente.
7         arrayValorsDobles[0] = 1;
8         //El resto se va llenando secuencialmente, a base de c lculos.
9         for(int i = 1; i < arrayValorsDobles.length; i++) {
10             arrayValorsDobles[i] = 2 * arrayValorsDobles[i - 1];
11         }
12         System.out.print("Se ha generado el array: [ ");
13         for (int i = 0; i < arrayValorsDobles.length; i++) {
14             System.out.print(arrayValorsDobles[i] +
15                             " ");
16         }
17         System.out.print("\n");
18     }
```

Para estudiar el funcionamiento del programa de forma m s detallada, resulta  til usar una tabla que indique la evoluci n de las variables en cada iteraci n, tal y como hace la tabla 1.2.

Tabla 1. 2. Evoluci n del bucle para inicializar un array con potencias de 2

Iteraci�n	Inicio del bucle	Condici�n vale	Posici�n modificada	Valor asignado
1	1	(1 < 20), true	arrayValorsDobles[1]	2*arrayValorsDobles[0], 2 1 = 2 *
2	2	(2 < 20), true	arrayValorsDobles[2]	2*arrayValorsDobles[1], 2 2 = 4 *
3	3	(3 < 20), true	arrayValorsDobles[3]	2*arrayValorsDobles[2], 2 4 = 8 *
4	4	(4 < 20), true	arrayValorsDobles[4]	2*arrayValorsDobles[3], 2 8 = 16 *
...				
18	19	(19 < 20), true	arrayValorsDobles[19]	2*arrayValorsDobles[18], 2*262144 = 524288
19	20	(20 < 20), false	Ya hemos salido del bucle	

Reto 1: haga un programa que inicialice proceduralmente un array con los 100 primeros n meros pares (Nota: el 0 es par).

1.4.2 Recorrido

Recorrido

Hablamos de recorrido cuando es necesario tratar todos los valores del array.

Cuando trabaje con arrays, a menudo tendrá que hacer cálculos en los que están implicados todos los valores almacenados o, al menos, buena parte. En este caso, es necesario realizar un recorrido secuencial de las posiciones e ir procesando para obtener un resultado final. Un ejemplo de una situación como ésta puede ser calcular la media aritmética de una serie de notas, que requiere sumar los valores de todas y cada una y dividir el resultado entre su número.

El caso de un array totalmente ocupado es el más sencillo, ya que es suficiente con recorrer el array en toda su extensión hasta llegar al final de la extensión, dada por la tamaño.

Como ejemplo, ejecute el siguiente código, que calcula la media aritmética de un seguido de notas, expresadas como valores reales, contenidas dentro de un array.

```
1 //Un programa que realiza un recorrido de un array: cálculo de la media aritmética.
2 public class RecorridoTotalmenteOcupado {
3     public static void main(String[] args) {
4         //Se parte de un array que contiene todos los valores.
5         float[] arrayNotes = {2f, 5.5f, 9f, 10f, 4.9f, 8f, 8.5f, 7f, 6.6f, 5f, 9f,
6             7f};
7         //Acumulador de la suma de valores.
8         float acumulado = 0;
9         //Hay que recorrer todo el array de extremo a extremo.
10        for(int i = 0; i < arrayNotes.length; i++) {
11            acumulado = acumulado + arrayNotes[i];
12        }
13        float resultado = acumulado / arrayNotes.length;
14        System.out.println("El resultado es" + resultado);
15    }
16 }
```

En la tabla 1.3, mediante el estudio de la evolución del bucle, se puede ver cómo se van recorriendo una por una todas las posiciones del array.

Tabla 1. 3. Evolución del bucle para recorrer el array y calcular la suma de todos los valores

Iteración	Inicio del bucle	Fin del bucle
	'y' vale	Condición vale
		Posición accedida
		'acumulado' vale
1	0	(0 < 12), true
2	1	(3 < 12), true
3	2	(4 < 12), true
...		
12	11	(11 < 12), true
13	12	(12 < 12), false

En el caso de un array parcialmente ocupado hay que tener cuidado, ya que el recorrido no necesariamente debe ser hasta la última posición, sino hasta la última posición ocupada. El algoritmo de recorrido a escala general será el mismo, pero la condición de salida del bucle va a cambiar. Por ejemplo, si el número de valores en el array se encuentra en la variable últimaPosicio, el código sería:


```
1 ...  
2 //Solo se tratan los valores realmente válidos.  
3 for(int i = 0; y < últimaPosicio; i++) {  
4 ...
```

Reto 2: modifique el código del recorrido para que, en lugar de la media, calcule cuál es la nota máxima.

1.4.3 Búsqueda

Otra tarea que a menudo se realiza al trabajar con arrays es buscar valores concretos en su interior. Por eso, es necesario ir mirando todas las posiciones hasta encontrarlo. Por ejemplo, ver si entre una serie de notas alguien ha sido capaz de sacar un 10.

Hasta cierto punto, se puede considerar que una búsqueda no es más que un tipo especial de recorrido. En este caso, el énfasis se hace en la circunstancia de que no siempre es necesario recorrer todos los elementos del array, sólo hasta encontrar el valor que buscamos. De hecho, llegar al final del array es lo que indica que no se ha encontrado el valor buscado.

Ejecute el siguiente código, que realiza la búsqueda de algún valor igual a 10 entre una serie de notas. Fíjese que en este código se sale del bucle en cuanto se encuentra el valor que buscamos. Ya no hace falta seguir iterando. Tal y como se presenta este código, el 10 se encuentra en la cuarta iteración. Puede ver qué pasa si no se encuentra ninguna si antes de compilar modifique manualmente este valor por otro en la inicialización del array.

Búsqueda

Hablamos de búsqueda cuando sólo es necesario tratar los valores del array hasta encontrar el objetivo.

```
1 //Un programa para ver si alguien ha sacado un 10.  
2 public class Búsqueda {  
3     public static void main(String args[]) {  
4         //Se parte de un array que contiene todos sus valores.  
5         float[] arrayNotes = {2f, 5.5f, 9f, 10f, 4.9f, 8f, 8.5f, 7f, 6.6f, 5f, 9f,  
6             7f};  
7         //Semáforo. ¿Se ha encontrado?  
8         boolean encontrado = false;  
9         //Contador de posiciones.  
10        int y = 0;  
11        //Se va mirando cada posición, mientras no se llega al final y no se encuentre un  
12        10.  
13        while ((y < arrayNotes.length)&&(!encontrado)) {  
14            if (arrayNotes[y] == 10) {  
15                encontrado = true;  
16            }  
17            e = i + 1;  
18        }  
19        //¿Se ha encontrado?  
20        if (encontrado) {  
21            System.out.println("Alguien ha sacado un 10.");  
22        } else {  
23            System.out.println("Nadie ha sacado un 10");  
24        }  
25    }  
26 }
```

Una vez más, una tabla para ver la evolución del bucle puede ayudarle a ver cómo, en cuanto se encuentra el valor buscado, las iteraciones terminan de inmediato. Cómo

se puede ver en la tabla 1.4, no es necesario recorrer todo el array. En la última iteración se subraya la parte de la expresión que causa que evalúe a false.

Tabla 1. 4. Evolución del bucle para recorrer el array y calcular la suma de todos los valores

Iteración	Inicio del bucle			Fin del bucle	
	'y' vale	'encontrado' vale	Condición vale	Posición accedida	'encontrado' vale
1	0	false	(0 < 12)&&(false), true	arrayNotes[0], 2.0	false
2	1	false	(1 < 12)&&(false), true	arrayNotes[1], 5.5	false
3	2	false	(2 < 12)&&(false), true	arrayNotes[2], 9.0	false
4	3	false	(3 < 12)&&(false), true	arrayNotes[3], 10.0	true
5	4	true	(4 < 12)&&(true), false	Ya hemos salido del bucle	

Reto 3: modifique el código del ejemplo para poder ver si hay alguien que debe suspendido la asignatura o no. Hágalo usando la sentencia break.

Si bien hasta el momento los esquemas de uso de arrays se han centrado en valores numéricos, también son de aplicación a otros tipos primitivos como los caracteres. Saber aplicarlos sobre conjuntos de caracteres es especialmente interesante para entender las estrategias para tratar texto. A título ilustrativo, a continuación se muestra un ejemplo de array basado en la búsqueda de un carácter concreto. Observe cómo el código relativo a la búsqueda, a grandes rasgos, es idéntico al relativo a valores numéricos. Sólo varía en los pequeños detalles relativos al tipo de dato que contiene el array.

```
1 //Buscar si existe la letra 'w' entre una serie de caracteres.
2 public class BúsquedaCaracter {
3     public static void main (String[] args) {
4         //Se parte de un array que contiene una serie de caracteres.
5         char[] arrayNotes = {'a', 'z', 'g', 'd', 'w', 'o', 'h', 'e', 'x', 's'};
6         //Semáforo. ¿Se ha encontrado?
7         boolean encontrado = false;
8         //Contador de posiciones.
9         int y = 0;
10        //Se va mirando cada posición, mientras no se llega al final y no se encuentre una
11        // "w".
12        while ((y < arrayNotes.length)&&(!encontrado)) {
13            if (arrayNotes[y] == 'w') {
14                encontrado = true;
15            }
16            e = i + 1;
17        }
18        //¿Se ha encontrado?
19        if (encontrado) {
20            System.out.println("La letra 'w' está en la lista.");
21        } else {
22            System.out.println("La letra 'w' no está en la lista.");
23        }
24    }
```

1.4.4 Uso de "arrays" auxiliares

La utilización de un array dentro del código de un programa puede ir más allá de almacenar secuencias de datos de longitud arbitraria obtenidas desde una entrada.

También son útiles como forma sencilla de gestionar muchos valores diferentes con un único identificador, en lugar de tener que declarar varias variables. Ésta vía es especialmente útil para casos en los que es necesario gestionar datos que tienen un cierto vínculo entre ellas. Otra utilidad de los arrays puede extraerse de su capacidad de acceder a sus posiciones usando un índice para simplificar el código.

Por ejemplo, suponga que desea hacer un programa que, a partir de las notas de los estudiantes de un aula, genere un gráfico de barras (o histograma) en el que se indique el número de estudiantes que han sacado suspendido, aprobado, notable o excelente. Para poder llevar a cabo esta tarea, necesitará disponer de una serie de contadores, uno para en cada categoría de nota, y realizar un recorrido de todas las notas. Según el rango en que pertenezca cada nota, se incrementará un contador u otro.

A continuación tiene el código del programa que hace esta tarea usando un array, en lugar de cuatro variables distintas. Compile y pruebe que funciona. En este caso, el array de notas ya está dado directamente, pero podría adaptar el código de forma que leyera los valores por el teclado.

```
1 public class Histograma {
2     public static void main (String[] args) {
3         float[] arrayNotes = {2f, 5f, 9f, 6.5f, 10f, 4.5f, 8.5f, 7f, 6f, 7.5f, 9f,
4             7f};
5         //Iniciación de los 10 contadores, para cada barra del gráfico.
6         int[] barras = new int[4];
7         //Cálculo de los contadores. Se realiza un recorrido de las notas.
8         for (int i = 0; i < arrayNotes.length; i++) {
9             //¿A qué rango pertenece?
10            if ((arrayNotes[i] >= 0) && (arrayNotes[i] < 5)) {
11                barras[0]++;
12            } else if (arrayNotes[i] < 6.5) {
13                barras[1]++;
14            } else if (arrayNotes[i] < 9) {
15                barras[2]++;
16            } else if (arrayNotes[i] <= 10) {
17                barras[3]++;
18            }
19            //Si no pertenece a ningún rango, nota incorrecta. La ignoramos.
20        }
21        System.out.println("Gráfico de barras de las notas");
22        System.out.println("-----");
23        //Se imprime el gráfico.
24        //Vamos recorriendo los contadores e imprimimos el valor en estrellas.
25        for (int i = 0; i < barras.length; i++) {
26            //Aprovechamos el índice para saber qué título es necesario escribir en la barra actual.
27            switch(i) {
28                case 0:
29                    System.out.print("Suspendido: ");
30                    break;
31                case 1:
32                    System.out.print("Aprobado: ");
33                    break;
34                case 2:
35                    System.out.print("Notable: ");
36                case 3:
37                    System.out.print("Excelente: ");
38            }
39        }
40    }
41}
```

```

37         System.out.print("Excelente: ");
38         break;
39     }
40     //Imprimimos tantas estrellas como el valor del contador.
41     for (int j = 0; j < barras[i]; j++) {
42         System.out.print("***");
43     }
44     System.out.println();
45 }
46 }
47 }

```

En este caso, la granularidad del gráfico es baja, por lo que se podría hacer con cuatro variables distintas sin problemas. Ahora bien, existe una parte del programa en el que disponer de un acceso a los cuatro valores por índice resulta de gran utilidad para simplificar el código: al escribir el gráfico por pantalla. Si se usaran cuatro variables, el código para imprimir el gráfico sería:

```

1 ...
2     System.out.println("Gráfico de barras de las notas");
3     System.out.println("-----");
4     //Se imprime el gráfico.
5     //Primero los suspendidos.
6     System.out.print("Suspendido: ");
7     for (int j = 0; j < contadorSuspes; j++) {
8         System.out.print("***");
9     }
10    System.out.println();
11    //Ahora los aprobados.
12    System.out.print("Aprobado: ");
13    for (int j = 0; j < contadorAprobados; j++) {
14        System.out.print("***");
15    }
16    System.out.println();
17    //Ahora los notables.
18    System.out.print("Notable: ");
19    for (int j = 0; j < contadorNotable; j++) {
20        System.out.print("***");
21    }
22    System.out.println();
23    //Ahora los excelentes.
24    System.out.print("Excelente: ");
25    for (int j = 0; j < contadorExcellent; j++) {
26        System.out.print("***");
27    }
28    System.out.println();
29    ...

```

Para escribir el resultado final en forma de gráfico es necesario escribir tantas estrellas como vale cada contador individual. Si se usan cuatro variables separadas, debería repetirse el mismo código para cada variable. Usando un array, basta con usar una estructura de repetición y realizar un recorrido. Además, el valor del índice al que se está accediendo también sirve para saber qué contador se está tratando y, por tanto, qué título es necesario escribir para cada iteración. Ahora imagináis que se quiere modificar el programa para contar la gente que ha sacado notas dentro de rangos con mayor granularidad (0-1, 1-2, ..., 9-10).

Por tanto, este es un ejemplo de caso en el que usar arrays auxiliares puede ser de grande utilidad para no tener que repetir el código.

1.4.5 Copia

Como se ha dicho, el tipo de dato array, al contrario que los tipos de datos primitivos, no tiene ninguna operación disponible. Sólo puede operarse en las posiciones individuales. Lo que sí está permitido es realizar asignaciones usando directamente los identificadores de las variables de tipo array. Ahora bien, un hecho ciertamente curioso es que la asignación no crea una copia de los datos del array original, sino que lo que hace es configurar el identificador de destino para que, a partir de entonces, por medio de éste se acceda a los mismos datos en memoria que el identificador origen.

La mejor forma de ver la diferencia de comportamiento entre una asignación entre tipos de datos primitivos y los arrays es con un ejemplo de código.

```
1 ...
2 //Asignación con tipos primitivos:
3 int y = 0;
4 int j = 5;
5 //A la variable "i" se le asigna el valor de "j".
6 e = j;
7 //Se modifica el valor de "j".
8 j = 10;
9 //i" conserva el valor copiado desde "j", 5.
10 System.out.println(i);
11 ...

1 ...
2 //Asignación con arrays:
3 int[] arrayA = {10, 20, 30, 40, 50};
4 int[] arrayB = {60, 70, 80, 90, 100};
5 //A la variable "arrayA" se le asigna el valor de "arrayB".
6 arrayA = arrayB;
7 //Se modifica el valor del índice 2 sólo de "arrayB".
8 arrayB[2] = 60;
9 //"arrayA" también ha visto modificado su valor en el índice 2!
10 System.out.println(arrayA[2]);
11 //Se modifica el valor del índice 4 sólo de "arrayA".
12 arrayA[4] = 70;
13 //"arrayB" también ha visto modificado su valor en el índice 4!
14 System.out.println(arrayB[4]);
15 //De hecho, "a" y "b" acceden exactamente a los mismos datos.
16 ...
```

La no creación de una copia real al llevar a cabo una asignación, sino dos vías de acceso a un mismo dato, es un rasgo común a todos los tipos de datos compuestos.

Una pregunta que puede surgir ahora es: ¿qué ha pasado con los datos que había inicialmente en la variable “arrayB”? La respuesta es que se han perdido. Ya no hay forma de acceder. En el caso de Java, esto no es ningún inconveniente, ya que cuando unos datos dejan de estar accesibles, se borran de la memoria y su espacio pasa a estar disponible para otras aplicaciones. En otros lenguajes, las implicaciones pueden ser distintos, o incluso se considera un error grave de programación.

Teniendo en cuenta el comportamiento de la asignación, para generar una copia independiente de un array es necesario tratar las posiciones como elementos individuales, como ocurría con el resto de operaciones.

Al realizar asignaciones entre variables de tipo array, el origen y destino ni tan siquiera sólo deben tener la misma tamaño.

```
1 //Un programa que crea una copia idéntica, e independiente, de otro array.
```

```
2 public class Copia {
3     public static void main (String[] args) {
```

```
4      float[] listaNotas = {2f, 5.5f, 9f, 10f, 4.9f, 8f, 8.5f, 7f, 6.6f, 5f, 9f,
5          7f};
6      //La copia debe tener el mismo tamaño que el original.
7      float[] copiaListaNotes = new float[listaNotes.length];
8      //Vamos copiando posición por posición.
9      for (int i = 0; i < listaNotes.length; i++) {
10         copiaListaNotes[i] = listaNotes[i];
11     }
12     //Mostramos la copia por pantalla.
13     System.out.print("El array copiado es: [ ");
14     for (int i = 0; i < listaNotes.length; i++) {
15         System.out.print(copiaListaNotes[i] + " ");
16     }
17     System.out.println("");
18 }
```

1.4.6 Cambio de tamaño

Una vez que se ha decidido por un valor concreto para el tamaño de un array, ya no hay vuelta atrás. No puede cambiarse. Esto tiene ciertos inconvenientes, ya que puede ocurrir que, en determinadas condiciones, se quede sin espacio para poder almacenar datos. Pero si bien no es posible cambiar el tamaño de un array, este problema no es irresoluble. Sólo hay que seguir los siguientes pasos:

1. Generar un nuevo array en una variable diferente, con un tamaño superior a el original.
2. Copiar los datos del array original en el mismo orden. Al hacerlo, el nuevo array quedará con posiciones "vacías".
3. Aprovechando las propiedades de la asignación entre arrays, se asigna el nuevo array en el array original.
4. Por último, añadir al nuevo array los valores extra que ya no cabían en el original.

Llegados a este punto, es como si a efectos prácticos se hubiera hecho mayor el array original. El siguiente código muestra un ejemplo de este esquema.

Un ejemplo de trozo de código que hace estas acciones sería el que se muestra en continuación. En éste, el tamaño del array original se incrementa de forma que se añaden MAX_VALORS, nuevas posiciones.

```
1 ...
2 float[] arrayNotes = new float[...];
3 ...
4 //PAS 1
5 float[] arrayNou = new float[arrayNotes.length + MAX_VALORS];
6 //PAS 2
7 for (int i = 0; i < arrayNotes.length; i++) {
8     arrayNou[i] = arrayNotes[i];
9 }
10 //PAS 3
11 arrayNotes = arrayNou;
12 //Ahora ya se pueden guardar MAX_VALORS nuevos valores extra dentro de "arrayNotes"
13 ...
```

El código siguiente adapta la lectura por teclado de una secuencia de longitud desconocida de forma que, si se supera la capacidad del array en el que se almacenan los valores, su tamaño se incrementa para poder albergar otros nuevos. Compílo y ejecútelo para ver cómo funciona.

```

1 importe java.util.Scanner;
2 public class CambiMida {
3     public static final int MAX_VALORS = 5;
4     public static void main (String[] args) {
5         //De entrada, el tamaño será 5.
6         //Se elige un valor muy bajo para que enseguida se fuerce el cambio de tamaño.
7         float[] arrayNotes = new float[MAX_VALORS];
8         //Es necesario un contador de posiciones en el que hay valores válidos.
9         int elementos = 0;
10        Scanner lector = new Scanner(System.in);
11        System.out.print("Ve escribiendo notas (valores reales entre 0 y 10).");
12        System.out.println("Puedes escribir varios en una sola línea, separados
            por espacios.");
13        //Será necesario alguna manera de saber que se ha terminado de escribir.
14        System.out.println("(Para terminar, escribe un -1)");
15        //Ir leyendo hasta encontrar la marca de fin. Pero el máximo es el tamaño del
            array.
16        //Si se entran más valores, cambiamos el tamaño del array y seguimos leyendo.
17        //Estructura de repetición con semáforo.
18        boolean marcaFi = false;
19        while (!marcaFi) {
20            //¿Hay un real?
21            if (lector.hasNextFloat()) {
22                //Hay que ver si es un valor válido o final de secuencia.
23                float nota = lector.nextFloat();
24                if ((nota >= 0)&&(nota <= 10)) {
25                    //Todo correcto. ¿Pero hay sitio en el array?
26                    if (elementos == arrayNotes.length) {
27                        //Hay que cambiar el tamaño del array. Se realiza 5 posiciones más largo.
28                        //Para ver cuándo sucede, avisamos por pantalla.
29                        System.out.println("Capacidad superada. Añadimos " + MAX_VALORS + "
                    posiciones más.");
30                        float[] arrayNou = new float[arrayNotes.length + MAX_VALORS];
31                        for (int i = 0; i < arrayNotes.length; i++) {
32                            arrayNou[i] = arrayNotes[i];
33                        }
34                        arrayNotes = arrayNou;
35                    }
36                    //Añadimos el nuevo valor. Seguro que hay sitio.
37                    arrayNotes[elementos] = nota;
38                    elementos++;
39                } else if (nota == -1) {
40                    //Marca de final, salimos del bucle.
41                    break;
42                }
43                //Si no es ni lo uno ni lo otro, lo ignoramos.
44            } else {
45                //Si no era un real, lo ignoramos.
46                lector.next();
47            }
48        }
49        //Ignoramos los valores sobrantes de la última línea.
50        lector.nextLine();
51        System.out.println("En el array hay almacenados " + elementos + " elementos
            : ");
52        for (int i = 0; i < elementos; i++) {
53            System.out.println(arrayNotes[i]);
54        }
55    }
56 }

```

Hay que decir que realizar este tipo de tarea es especialmente ineficiente y tiene un impacto sobre el rendimiento del programa. Es mejor sólo utilizar este esquema si realmente no hay más remedio. Si se hace, normalmente, a la hora de elegir el tamaño del nuevo array es mejor no ajustar demasiado y dejar espacio de sobra para llenar nuevas posiciones si es necesario sin tener que verte forzados a repetir este proceso constantemente.

1.4.7 Ordenación

En ocasiones resulta muy útil reorganizar los valores que hay dentro del array de forma que estén ordenados de manera ascendente o descendente. En muchos casos, mostrar los datos de forma ordenada es simplemente un requisito de la aplicación, de modo que el usuario pueda visualizarlos y extraer conclusiones de forma más cómoda. Pero existen casos en los que disponer de los datos ordenados facilita la codificación del programa y la eficiencia.

Por ejemplo, en este mismo apartado habéis visto cómo buscar, a partir de una serie de notas, si algún estudiante ha sacado un 10. Para realizar esta tarea, era necesaria una estructura de repetición e ir mirando uno por uno todos los valores dentro del array. Pero si los datos están ordenados en orden ascendente, entonces podemos utilizar una estrategia más eficaz. Si alguien tiene un 10, esta nota estará a la fuerza en la última posición. Por tanto, toda la búsqueda queda reducida directamente al código:

```
1 ...
2 if (arrayNotes[arrayNotes.length - 1] == 10) { System.out.println("Alguien ha
3     sacado un 10."); } else { System.out.println("Nadie ha sacado un
4     10");
5 }
6 }
7 ...
```

De repente el código se hace más rápido y corto, ya que no es necesaria la estructura de repetición. En general, los datos ordenados resultan de utilidad a la hora de realizar búsquedas o recorridos en los que los datos a tratar tienen un rango o un valor concreto, ya que permiten establecer estrategias que minimicen el número de iteraciones de los bucles.

Para ordenar una secuencia de valores existen varios algoritmos. Los más eficientes son bastante complejos y se encuentran más allá de los objetivos de este módulo. Basta con ver uno más sencillo.

El algoritmo de la burbuja (BubbleSort) sirve para ordenar elementos mediante recorridos sucesivos a lo largo de la secuencia, en los que se van comparando parejas de valores. Cada vez que se encuentran dos parejas en orden incorrecto, se intercambia su posición.

Básicamente, lo que se hace es buscar el valor más bajo, recorriendo todo el array, y ponerlo en la primera posición. Con esto ya tenemos la garantía de haber resuelto qué valor debe acabar en la primera posición. Entonces se busca el segundo valor más bajo, recorriendo la secuencia desde la segunda posición hasta el final, y se pone en la segunda posición. Y

la operación se repite hasta llegar a la última posición. Para realizar esta tarea se opera directamente sobre los valores almacenados en el array, intercambiándolos cada vez que encontramos un valor más bajo que el que hay en la posición que se está tratando.

El código sería el siguiente. Analícelo para entender qué está haciendo y pruebe que funciona correctamente:

```
1 //Un programa para ordenar valores usando el algoritmo de la burbuja.
2 public class Ordenar {
3     public static void main (String[] args) {
4         float[] listaNotas = {5.5f, 9f, 2f, 10f, 4.9f};
5         //Bucle externo.
6         //Se irá poniendo en cada posición tratada, empezando por la 0,
7         //el valor más bajo que se encuentre.
8         for (int i = 0; i < listaNotas.length - 1; i++) {
9             //Bucle interno.
10            //Se busca entre el resto de posiciones cuál es el valor más bajo.
11            for(int j = i + 1; j < listaNotas.length; j++) {
12                //La posición tratada tiene un valor mayor que el de la búsqueda. Los
                //intercambiamos.
13                if (listaNotas[i] > listaNotas[j]) {
14                    //Para intercambiar valores es necesaria una variable auxiliar.
15                    float cambio = listaNotas[i];
16                    listaNotas[i] = listaNotas[j];
17                    listaNotas[j] = cambio;
18                }
19            }
20        }
21        //Lo mostramos por pantalla.
22        System.out.print("El array ordenado es: ");
23        for (int i = 0; i < listaNotas.length;i++) {
24            System.out.print(listaNotas[i] +
25                               " ");
26        }
27        System.out.println("");
28    }
29 }
```

Dado que este algoritmo de ordenación se basa en dos bucles imbricados, vale la pena echar un vistazo más detallada a su funcionamiento. La tabla 1.5 muestra un seguido de iteraciones hasta tener en sus dos primeras posiciones los valores más bajos. Fíjese cómo el valor de la variable contador j siempre es superior a i. En las celdas en las que se indica el estado del array después de cada iteración, se subrayan las posiciones que se han intercambiado, en su caso.

Tabla 1. 5. Evolución del algoritmo de la burbuja

Inicio del bucle			Fin del bucle	
'i' vale	'j' vale	'listaNotas' vale	listaNotas[i] > listaNotas[j]	'listaNotas' vale
Inicio primera iteración bucle externo. Se busca el valor más pequeño para ponerlo en la posición 0.				
0	1	{5.5, 9, 2, 10, 4.9}	5.5 > 9, false	{5.5, 9, 2, 10, 4.9}
0	2	{5.5, 9, 2, 10, 4.9}	5.5 > 2, true	{2, 9, 5.5, 10, 4.9}
0	3	{2, 9, 5.5, 10, 4.9}	2 > 10, false	{2, 9, 5.5, 10, 4.9}
0	4	{2, 9, 5.5, 10, 4.9}	2 > 4.9, false	{2, 9, 5.5, 10, 4.9}
Fin primera iteración bucle externo. En listaNotas[0] se encuentra el valor más pequeño.				
Inicio segunda iteración bucle externo. Se busca el segundo valor más pequeño para ponerlo en la posición 1.				
1	2	{2, 9, 5.5, 10, 4.9}	9 > 5.5, true	{2, 5.5, 9, 10, 4.9}
.				

Tabla 1. 5 (continuación)

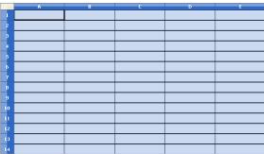
Inicio del bucle		Fin del bucle	
1	3	{2, 5.5, 9, 10, 4.9}	5.5 > 10, false {2, 5.5, 9, 10, 4.9}
1	4	{2, 5.5, 9, 10, 4.9}	5.5 > 4.9, true {2, 4.9, 9, 10, 5.5} —
Fino segunda iteración bucle externo. En listaNotes[1] se encuentra el segundo valor más pequeño.			
Inicio tercera iteración bucle externo. Se busca el tercer valor más pequeño para ponerlo en la posición 2.			
2	3	{2, 4.9, 9, 10, 5.5}	9 > 10, false {2, 4.9, 9, 10, 5.5}
2	4	{2, 4.9, 9, 10, 5.5}	9 > 5.5, true {2, 4.9, 5.5, 10, 9} — —
Fino tercero iteración bucle externo. En listaNotes[2] se encuentra el tercer valor más pequeño.			
Inicio cuarta iteración bucle externo. Se busca el cuarto valor menor para ponerlo en la posición 3.			
3	4	{2, 4.9, 5.5, 10, 9}	10 > 9, true {2, 4.9, 5.5, 9, 10} — —
Fino tercero iteración bucle externo. En listaNotes[3] se encuentra el cuarto valor más pequeño.			
Por la fuerza, en la posición 4 está el valor más alto.			
4	4	Hemos salido del bucle externo (4 > listaNotes.length - 1)	

Reto 4: modifique el ejemplo de ordenación de forma que, una vez ordenado el array, se calcule la media aritmética sólo de los estudiantes que han suspendido. El bucle del recorrido debe realizar exactamente tantas iteraciones como estudiantes suspendidos haya, y no como todo el tamaño del array. Reflexione atentamente sobre cómo el hecho de que el array esté ordenado es determinante para alcanzar esta última condición.

1.5 "Arrays" multidimensionales

Cuando se declara un array, existe una propiedad especial, aparte de su identificador de tipos y tamaño, que hasta el momento se ha obviado. Se trata de su dimensión, el número de índices a utilizar para establecer la posición a tratar. Todos los arrays con los que ha trabajado hasta ahora eran unidimensionales. Bastaba con un solo valor de índice para indicar la posición a acceder. Ahora bien, nada nos impide que el número de índices sea mayor que 1.

El orden de los índices es importante.



Un "array" de dimensión 2: una tabla

Un array multidimensional es aquél en el que para acceder a una posición concreta, en lugar de usar un solo valor como índice, se usa una secuencia de varios valores. Cada índice sirve como coordenada para una dimensión diferente.

Entre los arrays multidimensionales, el caso más usado es el de los arrays bidimensionales, de dos dimensiones. Éste es uno de los más fáciles de visualizar y es, por tanto, un buen punto de partida. Al tratarse de un tipo de array en el que se necesitan dos índices para establecer una posición, se puede considerar que su comportamiento es como el de una matriz, mesa u hoja de cálculo. El primer índice indicaría la fila, mientras que el segundo indicaría la columna donde se encuentra la posición a acceder.

Por ejemplo, suponga que el programa para tratar notas de estudiantes ahora debe gestionar las notas individuales de cada estudiante para una serie de ejercicios, de modo que se pueden realizar cálculos adicionales: calcular la nota final según los resultados individuales, ver la evolución de cada estudiante a lo largo del curso, etc. En este caso, toda esta información se puede resumir de forma eficiente en una tabla u hoja de cálculo, en la que los estudiantes se pueden organizar por filas, donde cada columna corresponde al valor de una prueba, y en la que la última columna puede ser la nota final.

1.5.1 Declaración de arrays bidimensionales

La sintaxis para declarar e inicializar un array de dos dimensiones es similar a la de los unidimensionales, si bien ahora debe especificarse que existe un índice adicional.

Esto se hace usando un par de claves extra, [], siempre que sea necesario especificar un nuevo índice.

Para declarar uno inicializado con todas las posiciones de sus valores por defecto, debería usarse la sintaxis:

```
1 palabraClauTipus[] identificadorVariable = new palabraClauTipus[númeroFilas][ númeroColumnas];
```

Como ocurría con los arrays unidireccionales, el valor de los índices, filas y columnas, no debe ser necesariamente un literal. Puede definirse de acuerdo con el contenido de una variable, diferente para cada ejecución del programa. El valor del número de filas y columnas tampoco debe ser necesariamente el mismo.

La inicialización mediante valores concretos implica indicar tantos valores como el número de filas por columnas. Para ello, primero se enumeran los valores individuales que hay en cada fila de la misma manera que se inicializa un array unidimensional: una secuencia de valores separados por comas y rodeados por corchetes, {...}.

Una vez enumeradas las filas de este modo, éstas se enumeran a su vez separadas por comas y rodeadas por corchetes. Conceptualmente, es como declarar un array de filas, en el que cada posición tiene otro array de valores. Esto se ve más claro con la sintaxis, que es la siguiente:

```
1 palabraClauTipus[][] identificadorVariable = { {Fila0valor1, Fila0valor2, ... , Fila0valorN},  
2                                             {Fila1valor1, Fila1valor2, ... , Fila1valorN},  
3  
4                                             ...,  
5                                             {FilaNvalor1, FilaNvalor2, ... , FilaNvalorN} };  
6
```

Fíjese cómo al inicio y al final están los corchetes extra que cierran la secuencia de filas y después de cada fila hay una coma, para separarlas entre sí. Para hacer más clara la lectura, se han usado diferentes líneas de texto para indicar los valores almacenados en cada fila, pero nada impide escribirlo todo en una sola línea muy larga. De todas formas, es recomendable usar este formato e intentar dejar espacios en los que corresponda de forma que los valores queden alineados verticalmente, ya que esto facilita la comprensión del código fuente.

Por ejemplo, para declarar e inicializar con valores concretos un array bidimensional de tres filas por cinco columnas de valores de tipo entero se haría:

```
1 int arrayBidi = {
2     {1, 2, 3, 4, 5},
3     {6, 7, 8, 9, 10},
4     {11, 12, 13, 14, 15}
5 };
```

Desde el punto de vista de una tabla o matriz, La primera fila sería {1, 2, 3, 4, 5}, la segunda {6, 7, 8, 9, 10} y la tercera {11, 12, 13, 14, 15}, tal y como muestra la tabla 1.6.

Tabla 1. 6. Resultado de inicializar con valores concretos uno array bidimensional de tres filas por cinco columnas

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15

Al usar este tipo de inicialización es necesario asegurarse de que los tamaños de todas las filas sean exactamente igual (que tengan el mismo número de valores separados entre comas), ya que es necesario garantizar que todas las filas tienen el mismo número de columnas.

1.5.2 Esquemas de uso de arrays bidimensionales

El acceso a las posiciones de un array bidimensional es prácticamente idéntico a el unidireccional, pero teniendo en cuenta que ahora hay dos índices por prever referentes a las dos coordenadas.

```
1 identificadorVariable[indiceFila][indiceColumna]
```

La tabla 1.7 hace un resumen de cómo se accedería a las diferentes posiciones de un array de 4 * 5 posiciones. En cualquier caso, al acceder, siempre es necesario mantener presente que cada índice nunca puede ser igual o superior al número de filas o columnas, según corresponda. De lo contrario, se produce un error.

Tabla 1. 7. Esta mesa misma es un array bidimensional de 4 * 5 posiciones

a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[0][4]
a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[1][4]
a[2][0]	a[2][1]	a[2][2]	a[2][3]	a[2][4]
a[3][0]	a[3][1]	a[3][2]	a[3][3]	a[3][4]

Asegúrese de usar variables diferentes por indicar cada índice.

Puesto que ahora hay dos índices, todos los esquemas fundamentales para trabajar con arrays bidimensionales se basan en dos estructuras de repetición imbricadas, una para tratar cada fila y la otra para tratar cada valor individual dentro de la fila.

En el caso de arrays bidimensionales, el atributo length tiene un comportamiento un poco especial, ya que ahora hay dos índices. Para entenderlo, hay que recordar que por inicializar el array lo que se hace es enumerar una lista de filas, dentro de las cuales existen los valores realmente almacenados. Por tanto, lo que le dice el atributo es el número de filas existentes.

Para saber el número de valores de una fila debe hacer:

```
1 identificadorVariable[indiceFila].length
```

Por ejemplo, si la tabla [1.7](#) corresponde a una variable llamada `arrayBidi`, `arrayBidi.length` evalúa a 4 (hay cuatro filas) y `arrayBidi[0].length`, `arrayBidi[1].length`, etc. todos evalúan a 5 (hay 5 columnas).

Inicialización procedural

De nuevo, se puede dar el caso en que desee asignar a cada posición valores que hay que calcular previamente y que, por tanto, haya que ir asignando uno por uno a todas las posiciones del array bidimensional.

A modo de ejemplo, compile y ejecute el código fuente siguiente, que almacena en cada posición la suma de los índices de un array bidimensional de dimensiones arbitrarias y después visualiza el resultado por pantalla, ordenado por filas.

```
1 importe java.util.Scanner;
2 //Un programa que inicializa un array bidimensional.
3 public class InicializacionBidi {
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         //Lee las filas.
7         int numeroFiles = 0;
8         while (numeroFiles <= 0) {
9             System.out.print("¿Cuántas filas tendrá la mesa? ");
10            if (lector.hasNextInt()) {
11                numeroFiles = lector.nextInt();
12            } else {
13                lector.next();
14                System.out.print("Este valor no es correcto. ");
15            }
16        }
17        lector.nextLine();
18        //Lee las columnas.
19        int numeroColumnas = 0;
20        while (numeroColumnas <= 0) {
21            System.out.print("¿Cuántas columnas tendrá la mesa? ");
22            if (lector.hasNextInt()) {
23                numeroColumnas = lector.nextInt();
24            } else {
25                lector.next();
26                System.out.print("Este valor no es correcto. ");
27            }
28        }
29        lector.nextLine();
30        //Inicialización con valores por defecto.
31        int[][] arrayBidi = new int[numeroFiles][numeroColumnas];
32        //Observe el uso del atributo "length".
33        System.out.println("Hay " + arrayBidi.length + " filas.");
34        + arrayBidi[0].length + " columnas.");
35        //Bucle para recorrer cada fila.
36        //"e" indica el número de fila.
37        for(int i = 0; i < numeroFiles; i++) {
38            //Bucle para recorrer cada posición dentro de la fila (columnas de la fila).
39            //"j" indica el número de columna.
40            for (int j = 0; j < numeroColumnas; j++) {
41                //Valor asignado a la posición: suma de los índices de fila y columna.
42                arrayBidi[i][j] = i + j;
43            }
44        }
45        //Se visualiza el resultado, también se necesitan dos bucles.
```

```

46     for(int i = 0; y < nombreFiles; i++) {
47         //Inicio de fila, abrimos corchetes.
48         System.out.print("Fila " + y + " { ");
49         for (int j = 0; j < númeroColumnas; j++) {
50             System.out.print(arrayBidi[i][j] + " ");
51         }
52         //Al final de cada fila se cierran corchetes y se da un salto de línea.
53         System.out.println("");
54     }
55 }
56 }

```

Recorrido

En el caso de arrays bidimensionales, también puede ocurrir que sea necesario realizar cálculos haciendo un recorrido por todos los valores contenidos. De nuevo, hay que ir fila por fila, mirando todas las posiciones de cada una.

Por ejemplo, suponga un programa en el que se guarda en cada fila el valor de las notas de los estudiantes de un curso. Se pide calcular la nota final de cada estudiante y almacenarla en la última columna, y también saber la media de todas las notas finales. El código se muestra a continuación; analízelo y pruebe que funciona. En este caso, para facilitar su ejecución, los valores de las notas están inicializados para valores concretos. En el caso de la nota final de cada estudiante, inicialmente se deja a 0 y será el programa quien la calcule.

Ahora sí: observe atentamente el uso del atributo `length` para controlar los índices en recorrer el array y saber cuál es el número de valores en una fila o el número de filas.

```

1 //Un programa que calcula notas medias en un array bidimensional.
2 public class RecorridoBidi {
3     public static void main (String[] args) {
4         //Datos de las notas.
5         float[][] arrayBidiNotes = {
6             { 4.5f, 6f 8f 0f },      , 5f      ,
7             { 10f 8f , 7.5f, 9.5f, 0f },
8             { 3f , 2.5f, 4.5f, 6f 0f },
9             { 6f , 8.5f, 6f 4f 0f },
10            { 9f , 7.5f, 7f 0f }      , 8f      ,
11        };
12        //Media aritmética del curso para todos los estudiantes.
13        float sumaFinales = 0f;
14        //Se hace tratando fila por fila, indicada por "i". Cada fila es un estudiante
15
16        // "arrayBidiNotes.length" evalúa el número de filas.
17        for(int i = 0; y < arrayBidiNotes.length; i++) {
18            //Aquí se acumulan las notas del estudiante tratado.
19            float sumaNotas = 0f;
20            //Tratamos cada fila (cada estudiante). Cada nota la indexa "j".
21            // "arrayBidiNotes[i].length" evalúa al número de posiciones de la fila.
22            for(int j = 0; j < arrayBidiNotes[i].length; j++) {
23                //¿Estamos en la última posición de la fila?
24                if(j != (arrayBidiNotes[i].length - 1)) {
25                    //Si no es la última posición, vamos acumulando valores.
26                    sumaNotas = sumaNotas + arrayBidiNotes[i][j];
27                } else {
28                    //Si lo es, es necesario escribir la media.
29                    //Hay tantas notas como el tamaño de una fila - 1.
30                    float notaFinal = sumaNotas/(arrayBidiNotes[i].length - 1);
31                    arrayBidiNotes[i][j] = notaFinal;
32                    System.out.println("El estudiante " + y + " ha sacado " + notaFinal + ".");
33                }
34            }
35        }
36    }
37 }

```

```
32         //Se actualiza la suma de medias de todos los estudiantes.
33         sumaFinales = sumaFinales + notaFinal;
34     }
35 }
36 //Fin del tratamiento de una fila.
37 }
38 //Fin del tratamiento de todas las filas.
39 //Se calcula la media: suma de notas finales dividido entre número de
    estudiantes.
40 float mediaFinal = sumaFinales / arrayBidiNotes.length;
41 System.out.println("La nota media del curso es " + mediaFinal);
42 }
43 }
```

Búsqueda

En arrays bidimensionales la búsqueda también implica tener que gestionar dos índices por ir avanzando por filas y columnas. Ahora bien, este caso es especialmente interesante, ya que al alcanzar el objetivo es necesario salir de las dos estructuras de repetición imbricadas. Por tanto, no se puede usar simplemente una sentencia break, ya que sólo sirve para salir de un único bucle. Es necesario tener un semáforo que se evalúe en ambos bucles.

Por ejemplo, suponga que el programa de gestión de notas quiere buscar si alguno estudiante ha sacado un 0 en alguno de los ejercicios a lo largo del curso. Habrá que ir mirando fila por fila todas las notas de cada estudiante, y cuando se encuentre un 0, terminar inmediatamente la búsqueda. El código para ello, basándose exclusivamente en un semáforo que dice si el valor ya se ha encontrado, sería el siguiente. Pruébalo en su entorno de trabajo.

En un caso como éste, hay que ir con mucho cuidado a inicializar e incrementar correctamente el índice para recorrer la posición de cada fila (es decir, que mira cada columna), ya que al usar una sentencia while en lugar de for no se hace automáticamente. Si se despista y no lo hace, el valor será incorrecto para sucesivas iteraciones del bucle que recorre las filas y el programa no actuará correctamente.

```
1 //Un programa que busca un 0 entre los valores de un array bidimensional.
2 public class BúsquedaBidi {
3     public static void main (String[] args) {
4         //Datos de las notas.
5         float[][] arrayBidiNotes = {
6             { 4.5f, 6f 5f 8f },      ,
7             { 10f 8f , 7.5f, 9.5f},
8             { 3f , 2.5f, 0f 6f },      ,
9             { 6f , 8.5f, 6f 4f },      ,
10            { 9f , 7.5f, 7f 8f }      ,
11        };
12        //Miraremos qué estudiante ha sacado el 0.
13        //Inicializamos a un valor inválido (de momento, ningún estudiante tiene un 0)
14        //Este valor hace de semáforo. Si toma un valor diferente, es necesario terminar la
        búsqueda.
15        int estudiante = -1;
16        //e indica la fila.
17        int y =0;
18        //Se va fila por fila.
19        //Se termina si se ha encontrado un 0 o si ya se ha buscado en todas las filas.
20        while ((estudiante == -1)&&(y < arrayBidiNotes.length)){
21            //j indica la "columna".
22            int j =0;
```

```

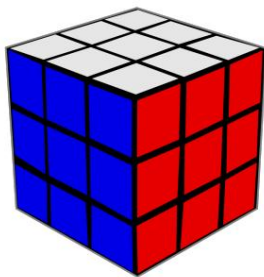
23 //Se busca en una fila.
24 //Se termina si se ha encontrado un 0 o si ya se ha buscado en todas las posiciones.
25 while ((estudiante == -1)&&(j < arrayBidiNotes[i].length)){
26     //¿Esta nota es un 0?
27     if (arrayBidiNotes[i][j] == 0f) {
28         //El índice que dice el estudiante es el de la fila.
29         estudiante = i;
30     }
31     //Se avanza a la siguiente posición dentro de la fila.
32     j++;
33 }
34 //Fin del tratamiento de una fila.
35 //Se adelanta a la siguiente fila.
36 i++;
37 }
38 //Fin del tratamiento de todas las filas.
39 if (estudiante == -1) {
40     System.out.println("Ningún estudiante tiene un 0.");
41 } else {
42     System.out.println("El estudiante" + estudiante + " tiene un 0.");
43 }
44 }
45 }

```

Reto 5: modifique el programa de forma que se busque si alguien ha sacado un 0, pero sólo en el segundo ejercicio (posición 1 de cada fila). Reflexione atentamente sobre si ahora realmente es necesario usar dos estructuras de repetición o no.

1.5.3 "Arrays" de más de dos dimensiones

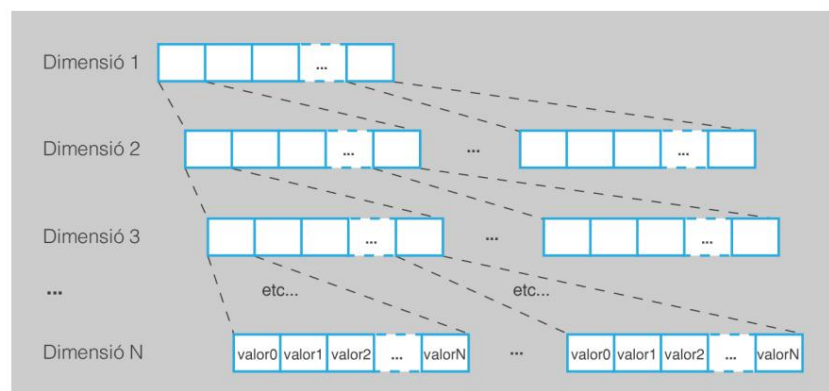
El caso de los arrays bidimensionales es el más común, pero nada impide que el número de índices necesarios para situar una posición sea también mayor de dos, de un valor arbitrario. Un array de tres dimensiones todavía se puede visualizar como una estructura tridimensional en forma de cubo, pero dimensiones superiores ya requieren más grado de abstracción, y por eso sólo se presentarán, sin entrar más detalle.



Un "array" con tres dimensiones: el cubo de Rubik. Imagen de Wikimedia Commons

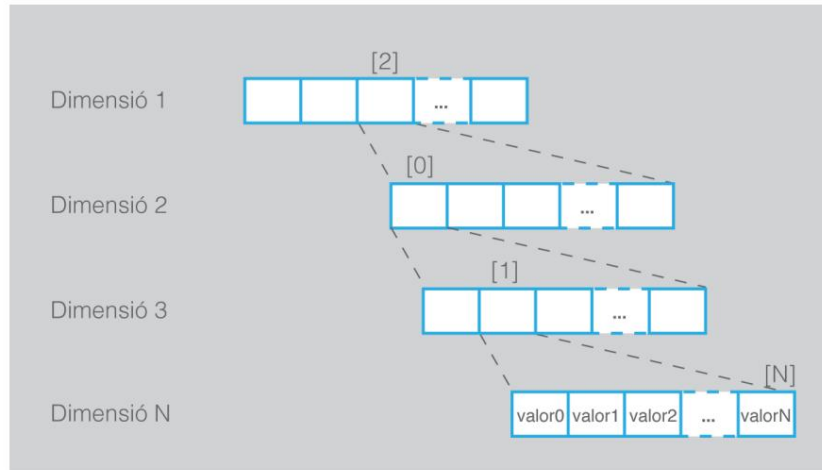
Conceptualmente, se puede considerar que definen una estructura de arrays imbricados a diferentes niveles como el que se muestra en la figura 1.7. Cada nivel corresponde a una dimensión distinta.

Figura 1. 7. Un "array" multidimensional: "arrays" dentro de "arrays" dentro de "arrays"...



Cada índice individual identifica una casilla del array elegido de una dimensión determinada, de modo que el conjunto de índices sirve como coordenada para establecer el valor final (que ya no será otro array) al que se desea acceder. La figura 1.8 muestra un ejemplo de cuatro dimensiones.

Figura 1. 8. Acceso a un "array" de cuatro dimensiones (hacen falta 4 índices)



Aunque esta estructura parece bastante compleja, no es muy diferente de una estructura de carpetas y archivos, donde las carpetas toman el papel de los arrays y los archivos el de los datos que se desean guardar. Puede tener una única carpeta para almacenar todas las fotos, que sería el caso de un único nivel de array (o sea, los arrays tal y como se han visto hasta ahora). Pero también podría organizarlas con una carpeta que a su vez contiene otras carpetas, dentro de las cuales se guardan diferentes fotos. Esto sería equivalente a disponer de dos niveles de arrays. Y así podría ir creando una estructura de carpetas con tantos niveles como quisiera. Para elegir un archivo basta con saber en qué orden es necesario ir abriendo las carpetas hasta llegar a la carpeta final, donde ya sólo necesita el orden del archivo.

Extendiendo las restricciones que cumplen los arrays bidireccionales al caso de los arrays multidimensionales:

- En una misma dimensión, todos los arrays tendrán exactamente la misma tamaño.
- El tipo de dato que se guarda en las posiciones de la última dimensión será el mismo para todos. Por ejemplo, no pueden tenerse reales y enteros mezclados.

En lenguaje Java, para añadir nuevas dimensiones por encima de la segunda, es necesario ir añadiendo claves [] adicionales a la declaración, una para cada dimensión.

```
1 palabraClauTipus[...][...] identificadorVariable = new palabraClauTipus[tamañoDim1
  ][...[tamañoDimN];
```

Por ejemplo, para declarar con valores por defecto un array de cuatro dimensiones, la primera de tamaño 5, la segunda de 10, la tercera de 4 y la cuarta de 20, sería necesaria la instrucción:

```
1 ...  
2 int[5][10][4][20] arrayQuatreDimensions = new int[5][10][4][20];  
3 ...
```

El acceso sería análogo a los arrays bidimensionales, pero usando cuatro índices rodeados por pares de llaves.

```
1 ...  
2 int valor = arrayQuatreDimensions[2][0][1][20];  
3 ...
```

En cualquier caso, se trata de estructuras complejas que no suelen usarse salvo en casos muy específicos.

1.6 Soluciones de los retos propuestos

Reto 1:

```
1 public class InicializaPares {  
2     public static void main(String[] args) {  
3         //Habr  que almacenar 100 enteros.  
4         int[] arrayPares = new int[100];  
5         //Vamos llenando cada posici n.  
6         for(int i = 0; i < arrayPares.length; i++) {  
7             arrayPares[i] = 2*i;  
8         }  
9     }  
10 }
```

Reto 2:

```
1 public class RecorridoMaxima {  
2     public static void main(String[] args) {  
3         //Se parte de un array que contiene todos sus valores.  
4         float[] arrayNotes = {2f, 5.5f, 9f, 10f, 4.9f, 8f, 8.5f, 7f, 6.6f, 5f, 9f,  
5             7f};  
6         //Valor m ximo hasta el momento.  
7         float notaMaxima = 0;  
8         //Hay que recorrer todo el array de extremo a extremo y guardar el mayor valor.  
9         for(int i = 0; i < arrayNotes.length; i++) {  
10             if (notaMaxima < arrayNotes[i]) {  
11                 notaMaxima = arrayNotes[i];  
12             }  
13         }  
14         System.out.println("La nota m xima es" + notaMaxima);  
15 }
```

Reto 3:

```
1 public class B squedaSuspas {  
2     public static void main (String[] args) {  
3         //Se parte de un array que contiene todos sus valores.  
4         float[] arrayNotes = {2f, 5.5f, 9f, 10f, 4.9f, 8f, 8.5f, 7f, 6.6f, 5f, 9f,  
5             7f};  
6         int y = 0;  
7         //Se va mirando cada posici n, mientras no se llega al final y no se encuentre un  
8             suspendido.  
9         //Se puede usar tanto "break" como "for".  
10        //Al encontrar un suspendido, directamente salimos del bucle.  
11        while (y < arrayNotes.length) {  
12            if (arrayNotes[i] < 5) {  
13                break;  
14            }  
15            i++;  
16        }  
17        if (y == arrayNotes.length) {  
18            //Si se ha superado el tama o del array, es que no se ha encontrado ning n suspendido.  
19            System.out.println("Nadie ha suspendido.");  
20        } else {  
21            //Si se ha salido antes de terminar el array, es que hab a un suspenso.  
22            System.out.println("Alguien ha suspendido.");  
23        }  
24    }  
25 }
```

Reto 4:

```
1 public class MediaSuspendidosOrdenados {
2     public static void main (String[] args) {
3         float[] listaNotas = {5.5f, 9f, 2f, 10f, 4.9f};
4         //Bucle externo.
5         //Se irá poniendo en cada posición tratada, empezando por la 0,
6         //el valor más bajo que se encuentre.
7         for (int i = 0; i < listaNotas.length - 1; i++) {
8             //Bucle interno.
9             //Se busca entre el resto de posiciones cuál es el valor más bajo.
10            for(int j = i + 1; j < listaNotas.length; j++) {
11                //La posición tratada tiene un valor mayor que el de la búsqueda; los
                //intercambiamos.
12                if (listaNotas[i] > listaNotas[j]) {
13                    //Para intercambiar valores es necesaria una variable auxiliar.
14                    float cambio = listaNotas[i];
15                    listaNotas[i] = listaNotas[j];
16                    listaNotas[j] = cambio;
17                }
18            }
19        }
20        //Lo mostramos por pantalla.
21        System.out.print("El array ordenado es: [");
22        for (int i = 0; i < listaNotas.length; i++) {
23            System.out.print(listaNotas[i] + " ");
24        }
25        System.out.println("");
26        //Inicio de las modificaciones.
27        //Valor acumulado para promediar.
28        float acumulado = 0;
29        //Número de suspendidos.
30        int numSuspendidos = 0;
31        //Inicio de la nueva parte.
32        //Como el array está ordenado, basta con hacer un recorrido hasta encontrar uno
        //aprobado,
33        //no es necesario llegar al final. Una vez se encuentra el primer aprobado, el resto de
34        //posiciones seguro que también son aprobados, puesto que el array está ordenado.
35        for (int i = 0; i < listaNotas.length; i++) {
36            //¿Es la nota de un aprobado?
37            if (listaNotas[i] >= 5) {
38                //Se ha encontrado el primer aprobado, ¡salimos del bucle!
39                break;
40            } else {
41                //Es un suspenso, vamos calculando.
42                acumulado = acumulado + listaNotas[i];
43                //Se incrementa el número de suspendidos.
44                numSuspendidos++;
45            }
46        }
47        float resultado = acumulado/numSuspendidos;
48        System.out.println("La nota media de los suspendidos es " + resultado);
49    }
50 }
```

Reto 5:

```
1 public class BúsquedaBidiZero {
2     public static void main (String[] args) {
3         //Datos de las notas.
4         float[][] arrayBidiNotes = {
5             { 4.5f, 6f { 10f 8f      , 5f      , 8f },
6             { 3f { 6f { 9f };      , 7.5f, 9.5f},
7             , 2.5f, 0f 6f },      ,
8             , 8.5f, 6f 4f },      ,
9             , 7.5f, 7f 8f }      ,
10
11         //Se necesitan dos estructuras de repetición si los dos índices deben ir
12         //variante para recorrer todas las posiciones. En este caso, un valor es
13         //siempre lo mismo (posición 1 de cada fila); sólo varía el número de fila.
14         //Por tanto, una única estructura es suficiente.
15         //Miraremos qué estudiante ha sacado el 0.
16         int estudiante = -1;
17         for (int numFila = 0; numFila < arrayBidiNotes.length; numFila++) {
18             if (arrayBidiNotes[numFila][1] == 0) {
19                 //Este estudiante tiene un 0.
20                 estudiante = numFila;
21                 //Ya podemos salir del bucle.
22                 //Lo podemos hacer, ya que sólo existe una única estructura de repetición.
23                 break;
24             }
25         }
26         if (estudiante == -1) {
27             System.out.println("Ningún estudiante tiene un 0.");
28         } else {
29             System.out.println("El estudiante " + estudiante + " tiene un 0 en el segundo
30         }
31     }
32 }
```


2. Tratamiento de cadenas de texto

Hasta ahora, el tipo de dato carácter no ha sido muy relevante a la hora de exponer ejemplos significativos de código fuente de programas. Es bastante habitual que en su día a día trate con conjuntos de datos numéricos individuales, cada uno independiente de los demás: fechas, distancias, intervalos de tiempo, cantidades, etc. Por tanto, son datos que tiene sentido almacenar y procesar mediante un programa para automatizar su cálculo. En cambio, seguramente no es muy habitual que use constantemente conjuntos de caracteres individuales e independientes: letras 'a', 'b', 'c', etc. La verdadera utilidad de los caracteres es poder utilizarlos agrupados en forma de palabras o frases: un nombre, una pregunta, un mensaje, etc. Por tanto, si un lenguaje de programación debe ser útil, debe tenerse en cuenta esta circunstancia.

El tipo de dato compuesto cadena de texto o String sirve para representar y gestionar de forma sencilla una secuencia de caracteres.

String

En Java, el tipo de dato compuesto cadena de texto se identifica con la palabra clave String.

2.1 La clase "String"

En lenguaje Java, la manipulación de cualquier dato de tipo compuesto, incluyendo las cadenas de texto, tiene una serie de particularidades y una terminología bastante diferente a la de los tipos primitivos o los arrays. Si bien no es necesario conocer todos sus detalles, antes de ver cómo manipular cadenas de texto u otros tipos de datos compuestos es importante aprender una serie de términos específicos, ya que aparecen con frecuencia en la documentación de Java.

El uso de los términos clase y objeto está estrechamente ligado a la característica de lenguaje Java de ser orientado a objetos.

En Java, exceptuando el caso del array, se usa el término clase para referirse a un tipo de dato compuesto.

A las variables de tipos de datos compuestos también podemos asignarles directamente valores, como a las de tipos primitivos. Ahora bien, con algunas particularidades en la nomenclatura.

Se utiliza el término objeto para referirse al valor asignado a una variable de un tipo de datos compuesto.

En lenguaje Java, la clase que representa una cadena de texto se llama explícitamente String. Así pues, a partir de ahora, en lugar de hablar del "tipo de dato compuesto String", diremos siempre "la clase String", y en lugar de decir que

Por convención de código, los nombres de las clases en Java se escriben siempre usando UpperCamelCase.

una variable "es del tipo de dato compuesto String" se dirá que "es de la clase o que pertenece a la clase String". Por lo general, también se hablará del "objeto asignado a la variable holaMon" en lugar de "el valor asignado a la variable holaMon", o de "un objeto de la clase String" en lugar de "un valor de tipo String".

En realidad, los conceptos de clase y objeto van mucho más allá de lo expuesto aquí, pero esto ya nos vale para poder explicar cómo manipular cadenas de texto de forma que se siga estrictamente la terminología Java tal y como se usa en sus fuentes de referencia.

2.1.1 Inicialización de objetos "String"

Trabajar con clases no modifica en absoluto la sintaxis para declarar una variable, pero sí puede tener algunas implicaciones a la hora de inicializarla, ya que es necesario asignar un objeto. Cómo se representa un objeto puede variar dependiendo de la clase empleada. Afortunadamente, en el caso de la clase String, la sintaxis para declarar e inicializar una variable es idéntica a cómo se haría con un tipo primitivo cualquiera, puesto que es posible representar un objeto mediante un literal directamente. Los literales asignables a una variable de la clase String toman la forma de cualquier combinación de texto rodeada entre comillas dobles, "...". Un ejemplo de declaración e inicialización de una variable que contiene una cadena de texto sería:

```
1 String holaMon = "¡Hola, mundo!";
```

Esta instrucción declara una variable llamada holaMon, perteneciente a la clase String, y le asigna el objeto que representa la cadena de texto "¡Hola, mundo!".

Vale la pena comentar que existe el concepto de cadena de texto "vacía", es decir, que no contiene absolutamente ningún carácter. Se representa simplemente no poniendo nada entre las comillas dobles. Expresado con código sería:

```
1 String cadenaVacía = "";
```

2.1.2 Manipulación de objetos

A diferencia de los tipos primitivos, cualquier objeto almacenado en una variable perteneciente a una clase (es decir, cualquier valor almacenado en una variable de un tipo de dato compuesto), no puede manipularse directamente mediante operadores de ningún tipo. No existen operadores que manipulen o comparen directamente objetos.

En algunos casos, la restricción en el uso de operaciones ya puede resultar evidente de forma intuitiva. Éste es el caso de los operadores aritméticos o lógicos. Por ejemplo, ¿con qué resultado evaluaría una división de cadenas de texto (objetos de la clase String) o su negación lógica? Ahora bien, en el caso de los operadores relacionales, aunque

sí puede tener sentido intentar comparar dos cadenas de texto, usarlos también es incorrecto. Por tanto, las siguientes expresiones no son válidas.

- `unString * unOtroString`
- `unString && unOtroString`
- `unString < unOtroString`
- `unString == unOtroString`
- etc.

En el caso de la igualdad hay que tener especial cuidado, ya que el compilador de Java no lo considera un error de sintaxis.

En Java, la manipulación de objetos se realiza mediante la invocación de métodos sobre las variables en las que se encuentran almacenados.

La lista de métodos que pueden ser invocados sobre una variable en la que se almacena un objeto depende exclusivamente de su clase. Clases distintas permiten la invocación de diferentes métodos. Por ejemplo, la clase `String` especifica el conjunto de métodos que es posible invocar sobre una variable a la que hay asignada una cadena de texto.

Para generar una instrucción en la que se llama un método, se usa el identificador de la variable, seguida de un punto, y entonces el nombre del método que queremos, escogido entre la lista que ofrece la clase a la que pertenece la variable. Dependiendo del método utilizado, puede ser necesario indicar una serie de información adicional, en forma de lista de valores entre paréntesis, separados por comas (... , ..., ...). Esta lista es lo que se llaman sus parámetros. Si el método no requiere ningún parámetro, basta con abrir y cerrar paréntesis, (). La sintaxis sería:

```
1 identificadorVariable.nombreMétodo(parámetro1, parámetro2,..., parámetroN);
```

Escribir una invocación de un método sobre una variable es equivalente a escribir una expresión que evalúa un cierto resultado. Cada método diferente realiza una operación o transformación diferente sobre el objeto almacenado en aquella variable. Para saber qué hace cada uno ya que tipo de dato pertenece su resultado, no queda más remedio que mirar la documentación de Java.

La invocación de un método tiene la máxima precedencia al evaluar una expresión.

La máxima precedencia de una invocación de un método tiene especialmente sentido en este caso, puesto que hasta que su resultado concreto no se ha evaluado, no se dispone de ningún valor a partir del cual se pueda continuar evaluando una expresión más compleja.

Por ejemplo, en el siguiente código, en el que se usa el método `length()`, que evalúa el tamaño de un objeto de la clase `String`, el resultado es 43, por el orden de precedencia

Puede encontrar la lista de métodos de la clase `String` en la web

<http://download.oracle.com/javase/6/docs/api/java/lang/String.html>.

Por convención de código, los nombres de los métodos de las clases de Java están escritos siempre usando `lowerCamelCase`.

de los operadores. Hasta que no se ha evaluado `length()` y se ha averiguado que el tamaño del texto es 22, ni siquiera sería posible iniciar el cálculo de la multiplicación. Esto evidencia que su precedencia debe ser la máxima.

```
1 public class Precedencia {  
2     public static void main (String[] args) {  
3         //El texto tiene un tamaño de 22 caracteres.  
4         String text = "Había una vez...";  
5         //orden: 1) método, 2) multiplicación, 3) resto.  
6         int dobleMidaMenysUn = 2 * text.length() - 1;  
7         System.out.println(dobleMidaMenysUn);  
8     }  
9 }
```

2.1.3 Acceso a los caracteres de una cadena de texto

Por la forma en que organizan los conjuntos de caracteres que contienen, las cadenas de texto son muy parecidas a los arrays. En el fondo, no dejan de ser una secuencia de tipos primitivos, en este caso caracteres (`char`), en los que cada letra ocupa una posición concreta. Por tanto, muchos de los aspectos generales aplicables a los arrays en Java también se pueden aplicar a las cadenas de texto. Nuevamente, el acceso se hace por el índice de la posición que ocupa cada carácter, comenzando desde 0 y hasta su longitud - 1.

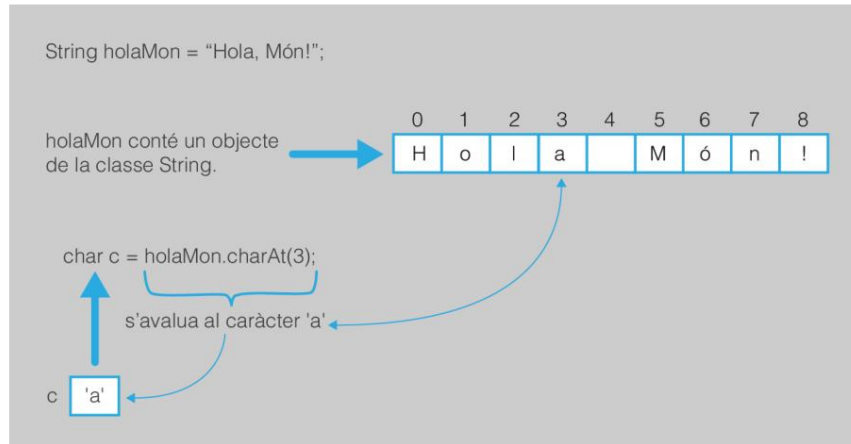
Ahora bien, dado que las cadenas de texto se almacenan en Java en variables de la clase `String` en forma de objetos, el acceso o manipulación de sus datos se debe realizar mediante la invocación de alguno de los métodos definidos en la clase `String`. Para el caso del acceso a caracteres individuales, el método a utilizar es el llamado `charAt(numPosición)`. Este método requiere un único parámetro de tipo entero en el que se indica qué posición se debe consultar, y evalúa igual el carácter en aquella posición.

Por ejemplo, ver qué carácter está en la tercera posición de una cadena de texto haría de la siguiente forma. Fíjese que la invocación de un método es equivalente a evaluar una expresión:

```
1 ...  
2 String holaMon = "¡Hola, mundo!";  
3 char c = holaMon.charAt(3);  
4 //En la variable "c" existe el carácter 'a'  
5 System.out.println("En la posición 3 está el carácter " + c);  
6 ...
```

La figura 2.1 muestra el comportamiento de la invocación en el método `charAt`. Fíjese que, en el fondo, éste es equivalente a evaluar una expresión. Su resultado puede ser asignado inmediatamente a una variable, o usado en otra expresión.

Figura 2. 1. Evaluación de la invocación del método charAt



Como con los arrays, la posición debe ser también un valor válido o se producirá un error. Para evitar esto, en Java las cadenas de texto también incorporan un mecanismo para establecer cuál es su longitud, similar al atributo length de los arrays. Se trata del método length(). Alerta, fíjese que, al tratarse de un método sin parámetros, inmediatamente al final lleva dos paréntesis, (). En el caso de los arrays, estos paréntesis no se utilizan.

Por ejemplo, el siguiente código escribe en líneas diferentes los caracteres individuales de la cadena de texto "¡Hola, mundo!". Pruebe que es así.

```

1 // Muestra los caracteres individuales en una cadena de texto. 2 public class MostraCaracters { public static void main
(String[] args) { String holaMon = "¡Hola, mundo!"; //Se recorren
3 las posiciones de la cadena de texto una por una. for (int i = 0; i < holaMon.length();
4 i++) { System.out.println(holaMon.charAt(i));
5
6
7
8 }
9 }
10 }

```

Por último, debe tener en cuenta que un punto muy importante del uso de los métodos es el hecho de que éstos se comportan exactamente igual que expresiones. Por tanto, no es posible realizar asignaciones sobre los mismos. En el caso de las variables de la clase String puede ser fácil confundirse, puesto que su comportamiento es muy parecido a los arrays, pero no del todo. Aunque si partís del funcionamiento de un array podría parecer lógico hacer lo siguiente, en realidad es un error. Sería como si, operando con valores enteros, intentara asignar un valor al resultado de una expresión suma:

```

1 ...
2 holaMon.charAt(3) = 'b' 3 //Erróneo. No es
realmente igual que hacer "array[3] = ...;".
4 //Sería más bien intentar hacer "(3 + 4) = a;".
5 ...

```

De hecho, la clase String Java no ofrece ningún método para cambiar el valor del carácter en una posición determinada. Reto 1: haga un programa que muestre por pantalla una cadena de texto, pero escrita del revés. Por ejemplo, si se parte de la cadena "¡Hola, mundo!", por pantalla debe mostrar "¡nómdro ,a!oH".

2.1.4 Concatenación de cadenas de texto

Si bien se ha dicho que los objetos no pueden manipularse mediante operadores, únicamente mediante la invocación de métodos, esto no es del todo cierto. Existe un único caso en el que sí es posible aplicar la operación de suma entre objetos. Se trata de la concatenación de cadenas de texto (objetos de la clase String). De hecho, éste es un caso realmente muy excepcional, ya que, si se fija, también se permite realizar esta operación entre cadenas de texto y datos de diferentes tipos primitivos, lo que en teoría tampoco está permitido. Ningún otro tipo de objetos permite en Java este tipo de comportamiento.

El resultado de la concatenación con cadenas de texto siempre da como resultado una nueva cadena de texto, que se puede utilizar tanto para generar mensajes mostrados directamente por pantalla como para realizar asignaciones a variables de la clase String.

Por tanto, recuerde que se puede hacer:

```
1 //Muestra el resultado de una división simple entre reales. 2 public class Dividir{ public static void main(String[]  
args){  
3  
4         double dividendo = 18954.74; double  
5         divisor = 549.12; double  
6         resultadoDivision = dividendo/divisor; //Se asigna el  
7         resultado de la concatenación a una variable de la clase  
            string.  
8         String mensaje = "El resultado obtenido es "          + resultadoDivision + ".";  
9         System.out.println(mensaje);  
10    }  
11 }
```

Debe tener presente que se encuentra ante una excepción, incorporada por los creadores de Java para facilitar su tarea como programadores a la hora de crear mensajes de texto.

2.1.5 "Arrays" de cadenas de texto

De la misma forma que se pueden declarar y usar arrays de tipo primitivos, también se pueden declarar y usar arrays de objetos, como las cadenas de texto. La sintaxis en este aspecto no varía en absoluto, si bien debe tener en cuenta que la parte de la instrucción vinculada a la definición del tipo indica ahora el nombre de la clase. Por ejemplo, para declarar un array con capacidad para 5 cadenas de texto se haría de la siguiente forma, según se usa el mecanismo con valores concretos o por defecto:

```
1 //Inicialización a valor concreto.  
2 //Un literal de una cadena de texto se representa con texto entre comillas dobles.  
3 String[] arrayCadenaConcret= {"Valor1", "Valor2", "Valor3", "Valor4", "Valor5"  
};  
4 //Inicialización a valor por defecto.  
5 String[] arrayCadenaDefecto = new String[5];
```

El caso de la inicialización a valores por defecto tiene una particularidad, y es que las posiciones toman un valor especial que indica que la posición está “vacía”, llamado null. Alerta, ya que este valor es un caso especial y no es equivalente a una cadena vacía o sin texto (representada con el literal ""). Mientras una posición está “vacía”, se cumple que:

- Si se muestra por pantalla, aparece el texto: “null”.
- Cualquier invocación de un método sobre él da un error.

Para acceder a una posición de un array de objetos, la sintaxis es igual que para los tipos primitivos, usando un índice entre claves ([i]). Sin embargo, los objetos tienen la particularidad de permitir la invocación de métodos. En este aspecto, una posición se comporta como una variable cualquiera. Esto puede verse en el ejemplo siguiente, especialmente en el segundo bucle, en el que se invoca el método length() directamente sobre la cadena de texto que hay en cada posición del array. Compílo y ejecútelo para averiguar qué hace:

```
1 public class ArrayString {  
2     public static void main(String[] args) {  
3         String[] texto = {"Hay", "había", "una", "veces..."};  
4         System.out.println("El texto del array es:");  
5         for (int i = 0; i < texto.length; i++) {  
6             System.out.println("Posición " + i + ": " + texto[i]);  
7         }  
8         System.out.println("Sus tamaños son:");  
9         for (int i = 0; i < texto.length; i++) {  
10            System.out.println("Posición " + i + ": " + texto[i].length());  
11        }  
12    }  
13 }
```

2.2 Entrada de cadenas de texto

De la misma manera que puede ser necesario introducir datos de tipos primitivos en el programa, de modo que se pueda trabajar, también puede ser necesaria la entrada de cadenas de texto. Para el caso de la entrada de cadenas de texto, sin embargo, tiene dos posibilidades.

Por un lado, se puede usar el teclado tal y como ha hecho hasta ahora. De hecho, ésta es la finalidad más natural para la que se emplea el teclado, ya que se trata de un periférico especialmente diseñado para escribir texto. Para ello es necesario utilizar las instrucciones de lectura que ya ha aprendido anteriormente, pero de modo que sea posible leer tanto palabras individuales como frases de texto completas.

Por otra parte, existe una posibilidad nueva: la entrada mediante argumentos en el método principal. Siempre que se ejecuta un programa en su ordenador es posible incluir directamente una serie de argumentos adicionales en forma de texto.

Si bien en los sistemas operativos modernos, basados en entornos gráficos de ventanas, esa circunstancia queda oculta, esta opción está presente. Java permite

averiguar si al ejecutar su programa se han incluido estos argumentos y, si es el caso, cuáles son.

2.2.1 Lectura desde teclado

El proceso de lectura de cadenas de texto desde teclado es muy parecido a la lectura de tipos primitivos. Basta con saber cuáles son las instrucciones adecuadas para obtener cadenas de texto desde teclado, en este caso en forma de objetos de la clase String. Antes de ver cuáles son estas instrucciones, sin embargo, vale la pena hacer una pequeña revisión de los elementos que intervienen en el proceso de lectura de datos por teclado aprovechando que se han introducido los conceptos de clase, objeto y método, bajo una nueva perspectiva.

Revisión de la lectura por teclado

Vuelva a echar un vistazo a un ejemplo simple de lectura de datos por teclado. Por ejemplo, un programa que lee un valor entero y lo muestra por pantalla, comprobando antes de que el tipo del valor escrito sea el correcto.

```
1 importe java.util.Scanner;  
2 //Lee un entero, comprobando que sea correcto.  
3 public class LecturaEnter {  
4     public static void main (String[] args) {  
5         Scanner lector = new Scanner(System.in);  
6         int valor = 0;  
7         System.out.print("Escribe un valor entero: ");  
8         if (lector.hasNextInt()) {  
9             valor = lector.nextInt();  
10            System.out.println("El valor era " + valor + ".");  
11  
12            lector.next();  
13            System.out.print("El valor no era entero.");  
14        }  
15        lector.nextLine();  
16    }  
17 }
```

Este código tiene las siguientes particularidades:

- Scanner es una clase. Ésta ofrece métodos para leer datos desde el teclado.
- lector es una variable en la que se almacena un objeto perteneciente a esta clase.
- hasNextInt(), nextInt() y next() son métodos, ofrecidos por la clase Scanner. De hecho, todas las instrucciones next... y hasNext... vinculadas en la lectura de datos por el teclado son métodos de esta clase. Al invocarlos, provocan que se haga una lectura desde el teclado y evalúan el dato leído. Según el tipo de dato que se desea leer, es necesario usar un método u otro.

El único punto que todavía es una incógnita ahora mismo es cómo funciona la inicialización de una variable de la clase Scanner, de modo que contenga un objeto. No es necesario que lo entienda, simplemente basta con saber que para esta clase se hace así. Ahora bien, es importante tener en cuenta que sin una correcta inicialización de la variable lector con objeto la lectura de datos no será correcta.

Lectura de palabras individuales

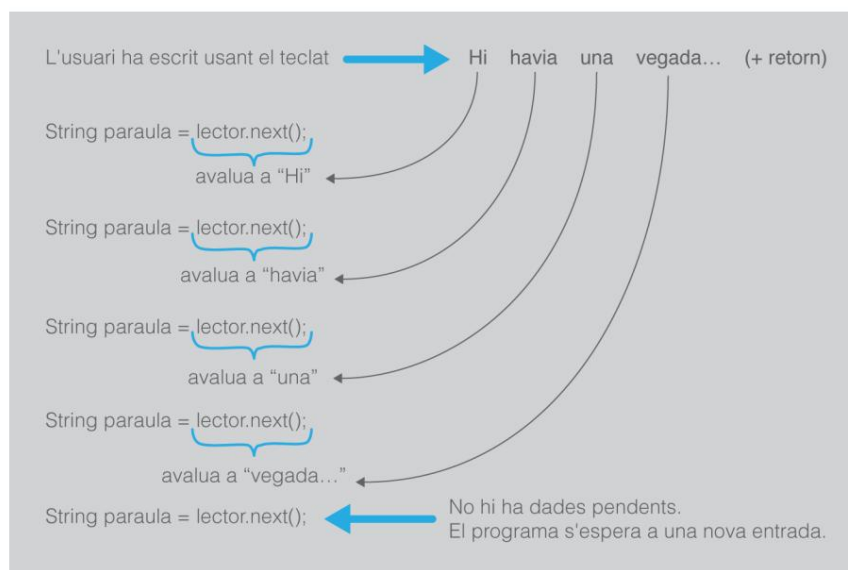
Al igual que hasta ahora se han leído tipos primitivos individuales entrados por teclado, también es posible leer palabras.

El método de la clase Scanner vinculado a la lectura de una cadena de texto compuesta de una única palabra es `next()`.

Su comportamiento es exactamente lo que ha visto hasta el momento para leer tipos primitivos, con la única diferencia de que la invocación de este método sobre un objeto de la clase Scanner evalúa una cadena de texto. Recuerde que si en una misma línea escriba más de una palabra, sucesivas invocaciones a este método no bloquearán el programa, sino que irán evaluando los sucesivos datos pendientes de leer. Este comportamiento queda esquematizado en la figura 2.2.

Debe tenerse en cuenta que, en este contexto, cuando se habla de palabra, en realidad se está refiriendo a cualquier combinación de letras o números que no contiene ningún espacio en medio. Por tanto, este método puede leer los textos individuales "Hola,", "mundo!", "1024", "3.1426", "base10", etc. Ahora bien, el resultado de una lectura usando este método siempre evalúa una cadena de texto, nunca un tipo numérico. Incluso cuando el valor leído es un número.

Figura 2. 2. Lectura de secuencias de palabras



Un hecho remarcable en el caso de la entrada de cadenas de texto desde teclado es que no es necesario realizar ninguna comprobación de tipo previa a la lectura (métodos hasNext...) , ya que todo dato escrito mediante el teclado siempre se puede interpretar como una cadena de texto. Nunca puede producirse un error en este sentido.

La mejor forma de ver su comportamiento es con un ejemplo. Compile y ejecute el programa siguiente, que lee hasta 10 palabras y las va mostrando por pantalla. Intente escribir más de una palabra en cada línea (una frase entera) y observa cómo las lee todas una por una antes de esperar nuevamente una entrada de datos. Fíjese que las palabras quedan discriminadas en la lectura, al estar separadas con espacios.

```
1 importe java.util.Scanner;
2 //Lee un cierto número de palabras (en este caso, 10).
3 public class LecturaPalabras {
4     public static final int NUM_PALABRAS = 10;
5     public static void main (String[] args) {
6         Scanner lector = new Scanner(System.in);
7         System.out.println("Escribe" + NUM_PALABRAS + " palabras separadas por
            espacios.");
8         System.out.println("Las puedes escribir en líneas de texto diferente, si quieres."
            );
9         for (int i = 0; i < NUM_PALABRAS; i++) {
10             //Se van leyendo palabras una por una.
11             //Recordar el comportamiento lectura de secuencias de datos por el teclado.
12             String palabra = lector.next();
13             System.out.println("Palabra " + i + ": Has escrito \"" + palabra + "\".");
14         }
15         //Se lee el resto de la línea y se ignora el contenido.
16         lector.nextLine();
17     }
18 }
```

Lectura de caracteres individuales

Hay casos en los que sólo se quiere leer un carácter, para simplificar la entrada de datos. Por ejemplo, como sistema abreviado para elegir opciones de un menú, o en un programa en el que se den diferentes opciones etiquetadas con letras individuales (cómo podría ser resolver una pregunta tipo test). En un caso como éste, y dado que la clase Scanner no ofrece ningún método para leer caracteres individuales, lo que hay que hacer es leer la entrada como una cadena de texto, comprobar que ésta sólo se compone de un único carácter y extraerlo.

- Para leer una cadena de texto existe el método next(), como apenas se ha visto.
- Para establecer la longitud de una cadena de texto se dispone del método length(). En este caso, ver si es 1.
- Para extraer un carácter individual, tenemos el método charAt(numPosición). En este caso, la posición a consultar es la 0.

En el siguiente programa de ejemplo hay que escoger la respuesta correcta entre cuatro opciones etiquetadas con una letra. Para simplificar el código, sólo se permite un solo intento. Ejecutarlo y observa atentamente cada uno de los pasos apenas descritas para ver si realmente se ha leído una única letra y extraerla.


```
1 importe java.util.Scanner;
2 // Muestra una pregunta tipo test y mira si se adivina.
3 public class LecturaCaracter{
4     public static final char RESPUESTA_CORRECTA = 'b';
5     public static void main (String[] args) {
6         Scanner lector = new Scanner(System.in);
7         System.out.println("Adivina la pregunta.");
8         System.out.println("¿Cuál de los siguientes no es un tipo primitivo?");
9         System.out.println("a) Enter");
10        System.out.println("b) Scanner");
11        System.out.println("c) Carácter");
12        System.out.println("d) Booleano");
13        System.out.print("Tu respuesta es la opción: ");
14        //Se lee la cadena de texto.
15        String palabra = lector.next();
16        //¿Es una palabra de un único carácter?
17        if (palabra.length() == 1) {
18            //Se extrae el carácter de la cadena de texto.
19            char caracter = palabra.charAt(0);
20            //¿Es un carácter válido? (a, b, cod)
21            if ((caracter >= 'a') && (caracter <= 'd')) {
22                //¿La respuesta final es correcta?
23                if (caracter == RESPUESTA_CORRECTA) {
24                    System.out.println("Efectivamente, la respuesta era " +
25                        RESPUESTA_CORRECTA + ".");
26                } else {
27                    System.out.println("La respuesta " + caracter + " es incorrecta.");
28                }
29            } else {
30                System.out.println("'" + caracter + "' es una opción incorrecta.");
31            }
32        } else {
33            //No lo era.
34            System.out.println("'" + palabra + "' no es un carácter individual.");
35        }
36        lector.nextLine();
37    }
```

Reto 2: modifique el programa anterior para permitir hasta tres intentos de respuesta en lugar de uno.

Lectura de frases enteras

Hasta el momento, sólo ha sido posible leer elementos individuales, uno por uno y separados por espacios, dentro de una secuencia entrada por teclado. Ahora bien, a menudo es útil leer frases enteras, compuestas por un conjunto de palabras separadas por espacios. Por ejemplo, entrar un nombre y apellidos, o una dirección postal. En este caso, quisiéramos guardar toda la cadena de texto dentro de una única variable.

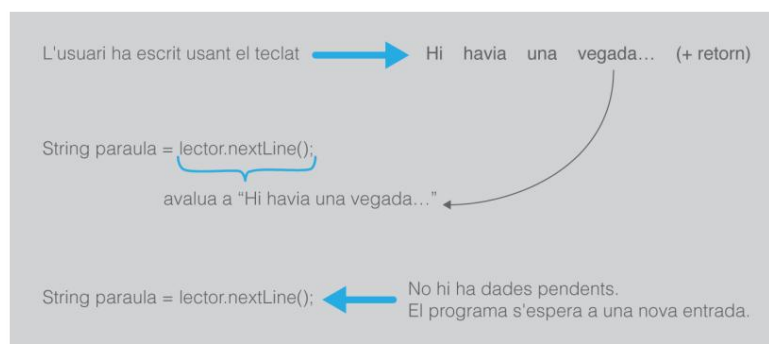
El método de la clase Scanner vinculado a la lectura de una cadena de texto en forma de frase en la que hay varias palabras separadas por espacios es `nextLine()`.

Hasta el momento este método ha sido usado para descartar toda la lista de elementos pendientes de leer del teclado (en caso de que se haya escrito más de un elemento en una única línea). En realidad, este método lo que hace es leer todos los elementos pendientes que leer en la línea actual y evaluar la cadena de texto que los contiene todos. Por tanto,

sirve para leer conjuntos de elementos de golpe, independientemente de los espacios en blanco que les separan.

A efectos prácticos con vistas a la lectura de cadenas de texto, si alguna vez se combina el uso de `nextLine()` con otros métodos de lectura de elementos individuales (`next()`, `nextInt()`, etc.) es posible usarlo para leer frases enteras directamente. Cada vez que se invoca, el programa se bloquea y espera a que el usuario entre una frase entera y pulse la tecla de regreso. Entonces, evalúa toda la frase escrita (todas las palabras con espacios incluidos). En este caso, y al contrario que con la resto de métodos de lectura, no es posible escribir más de un elemento dentro de una misma línea de texto, ya que lee toda la línea al completo desde la última lectura hasta encontrar un salto de línea. Este comportamiento se esquematiza en la figura 2.3. Contraste con el mostrado en la figura 2.2.

Figura 2. 3. Lectura de frases



El siguiente programa de ejemplo muestra cómo se lee una línea de texto completa escrita por el teclado, de forma muy parecida al ejemplo anterior de leer palabras individuales. Sin embargo, fíjese que hay algunas diferencias en el código y en el comportamiento. Concretamente, nunca se dará el caso de que una lectura por el teclado no ocasione el bloqueo del programa a la espera de que escriba algo. Absolutamente siempre se detendrá a esperar a que escriba un texto y pulse la tecla de retorno. Pruebe el programa para comprobar que es así.

```

1 importe java.util.Scanner;
2 //Lee frases escritas por el teclado.
3 public class LecturaFrase {
4     public static final int NUM_FRASES = 4;
5     public static void main (String[] args) {
6         Scanner lector = new Scanner(System.in);
7         System.out.println("Escribe" + NUM_FRASES + "frases.");
8         System.out.println("Para terminar una frase, pulsa la tecla de retorno.");
9         for (int i = 0; i < NUM_FRASES; i++) {
10             //Se van leyendo frases una por una.
11             String frase = lector.nextLine();
12             System.out.println("Frase " + i + ": Has escrito \"" + frase + "\".");
13         }
14         //Ahora no hace falta leer el resto de ninguna línea, ya que siempre se leen
15         //líneas enteras...
16     }
17 }

```

2.2.2 Argumento del método principal

Esta opción permite establecer una entrada de datos en el programa sin tener que interrumpir su ejecución para preguntar algo al usuario y esperar una entrada por teclado.

Una pregunta que se puede plantear es cuál es la utilidad de este sistema cuando ya está la entrada por el teclado. Un ejemplo de situación en que esto es muy útil es en programas que se comportarán de forma diferente según una entrada de datos, pero en los que no se quiere, o no se puede, esperar a que el usuario esté pendiente. Por ejemplo, suponga un programa que desea que se ejecute cada noche para realizar copias de seguridad de una serie de archivos. Por un lado, no tiene sentido que el programa esté siempre en marcha comprobando si ha llegado la hora de realizar la copia. Es más lógico programar su ejecución periódica con alguna herramienta ofrecida por el sistema operativo. Ahora bien, es evidente que no es factible que alguien esté pendiente del teclado cada vez que se ejecuta, pero para que funcione correctamente debe disponer de cierta información que tampoco puede estar escrita en el código fuente, ya que el valor dependerá de cada usuario que lo utilice: la lista de directorios que se desea copiar, donde se guarda la copia, etc. Por tanto, es útil disponer de una manera simple de entrar datos en un programa sin que tenga que ser siempre por el teclado.

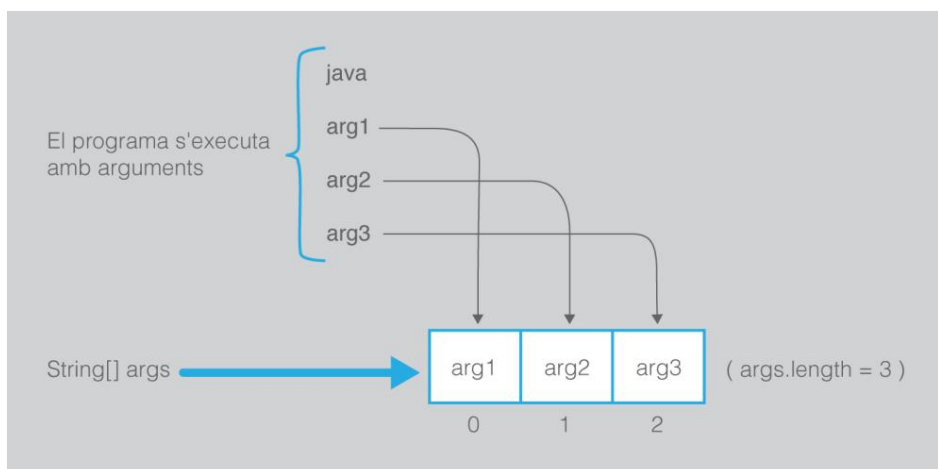
Ejemplos de herramientas que permiten la ejecución diferida de programas son las tareas programadas en Windows o la instrucción cron de Unix.

Normalmente, la lista de argumentos de un programa se introduce como un conjunto de palabras a continuación del nombre del archivo ejecutable, separadas por espacios.

La lista de palabras que conforman los argumentos de un programa en Java puede obtenerse directamente a partir de la variable especial que se define en la declaración del método principal: `String[] args`. Concretamente, toma la forma de un array de cadenas de texto, en el que éstas ocupan las posiciones en el mismo orden en que se han escrito los argumentos al ejecutar el programa, tal y como muestra la figura 2.4.

Dispone de un anexo en el que le explicamos cómo incluir argumentos al ejecutar un programa mediante el IDE que usará a lo largo del curso.

Figura 2. 4. Asignación de argumentos de entrada en la ejecución del programa "java" a la variable "args"



Ejecutar un programa con argumentos

Normalmente, los ejemplos básicos de programas ejecutados con argumentos son los vinculados en la línea de órdenes. Entre ellos, se puede encontrar el intérprete del propio Java, el ejecutable `java` (en sistemas Unix) o `java.exe` (en sistemas Windows). Si abre una ventana de línea de comandos y escribe `java -help`, aparece una lista de los argumentos posibles para ejecutarlo sin la necesidad de un IDE: los archivos con bytecode Java que debe ejecutar, si es necesario mostrar su versión, si debe mostrar información de depuración adicional, etc. De hecho, `-help` es un argumento en sí mismo.

Su IDE automatiza la ejecución y el paso de argumentos a este programa sin la necesidad de tener que preguntar nada. Partiendo de los archivos que ha generado, él solo genera la lista de argumentos a utilizar.

Para saber con cuántos argumentos se ha ejecutado el programa es necesario usar el atributo `length` asociado a los arrays. Si no se ha pasado ningún argumento, su valor será 0. Tenga cuidado con esta situación, ya que entonces cualquier acceso a una posición del array será siempre incorrecto.

A modo de ejemplo, pruebe el siguiente programa, que enumera los argumentos con los cuales se ha ejecutado. A efectos prácticos, se trata de un recorrido sobre un array, si bien los elementos almacenados en sus posiciones ahora son cadenas de texto en sitio de valor numéricos.

```
1 //Programa que escribe por pantalla los argumentos de un programa.
2 public class LecturaArgumentos {
3     //Los argumentos están en la variable "args".
4     public static void main(String[] args) {
5         //Primero hay que mirar si hay alguno.
6         if (args.length > 0) {
7             //Los hay. Se muestran por pantalla ordenadamente.
8             for (int i = 0; i < args.length; i++) {
9                 System.out.println("Argumento " + i + ": " + args[i]);
10            }
11        } else {
12            //No hay ninguna.
13            System.out.println("No hay ningún argumento.");
14        }
15    }
16 }
```

2.3 Manipulación de cadenas de texto

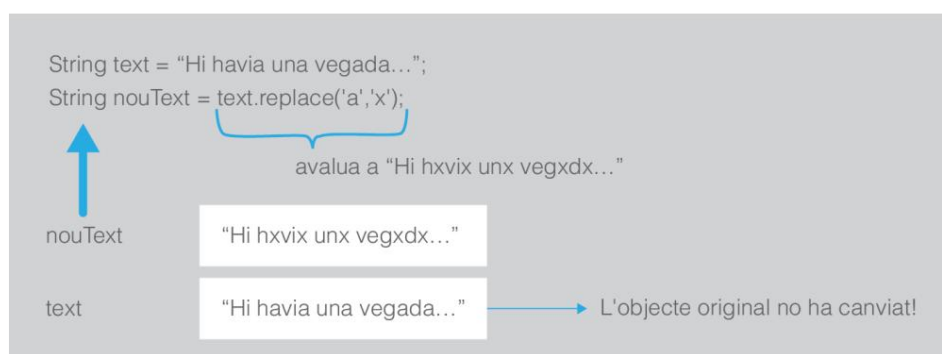
Aunque una cadena de texto organiza los caracteres que almacena de una modo muy parecido a cómo lo haría un array, en forma de secuencia ordenada en la que cada elemento ocupa una posición identificada por un índice, hay dos diferencias destacadas en su comportamiento y que tienen gran incidencia a la hora manipularlas.

Por un lado, la clase `String` ofrece un abanico muy amplio de métodos con los que se pueden manipular, por lo que en la inmensa mayoría de casos no será necesario realizar un tratamiento individual de cada posición (por ejemplo, mediante estructuras de repetición). Normalmente, todos los esquemas de uso y manipulación de cadenas de texto se basan simplemente en la invocación del método adecuado.

Por otra parte, otra particularidad que las diferencia de los arrays es que son inmutables. La invocación de un método puede consultar los datos dentro de una cadena de texto, o puede generar una nueva cadena de texto con ciertas modificaciones respecto al original. Sin embargo, nunca se verá modificado el valor inicial de la cadena de texto original. Si queremos, esta nueva cadena puede ser asignada a la variable original, por lo que se reemplaza el valor inicial por el nuevo.

La figura 2.5 muestra este comportamiento por la invocación del método `replace(carácterNou, carácterVell)`, que reemplaza todas las apariciones de un carácter (`carácterVell`) por otro diferente (`carácterNou`).

Figura 2. 5. Los métodos invocados sobre cadenas de texto no modifican su valor, crean nuevas modificadas



2.3.1 Búsqueda

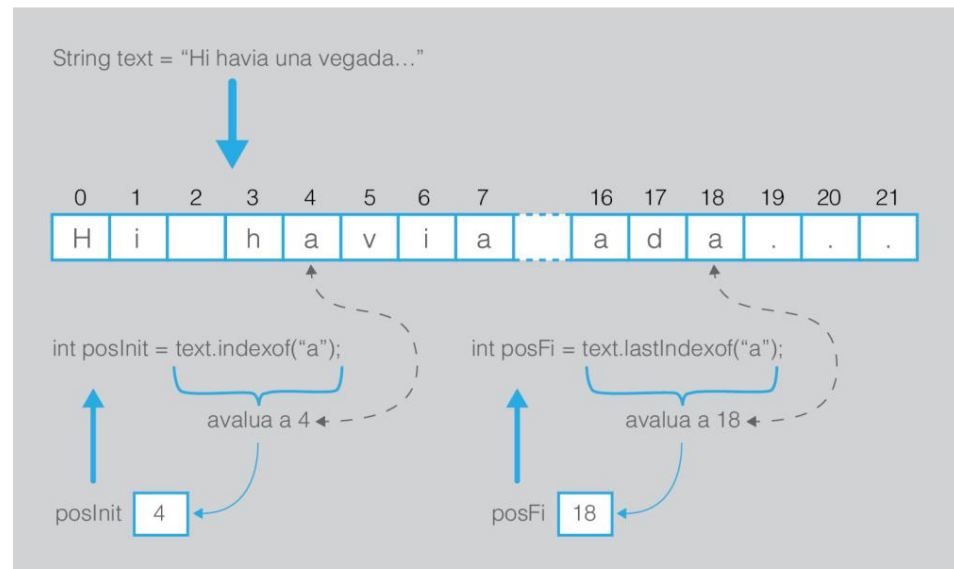
Normalmente, el proceso de búsqueda de un valor concreto dentro de un array debería realizarse con una estructura de repetición que fuese comprobando si ese valor existe en alguna de las posiciones, hasta encontrarlo o llegar al final del array. Esto podría replicarse con las cadenas de texto usando el método `charAt`. Por suerte, la clase `String` ofrece dos métodos que ahorran esta tarea, ya que evalúan directamente la posición en la que se encuentra un carácter o una subcadena de texto concreta.

- `indexOf(textABuscar)` evalúa la primera posición dentro de la cadena de texto donde se encuentra el texto a buscar.
- `lastIndexOf(textABuscar)` evalúa la última posición dentro de la cadena de texto donde se encuentra el texto a buscar.

En ambos casos, `carácterA Buscar` siempre debe ser una cadena de texto (ya sea mediante un literal o una variable) y si el texto buscado no existe, evalúan a -1. Por tanto, no sólo es posible saber si existe el valor buscado, sino también establecer directamente posiciones donde se puede encontrar.

La figura 2.6 muestra el comportamiento de ambos métodos si se busca el carácter 'a' en la frase "Érase una vez...".

Figura 2. 6. Comportamiento de los métodos "indexOf" y "lastIndexOf"



El siguiente código muestra un ejemplo de búsqueda de un carácter dentro de un texto cualquiera (compuesto por varias palabras). Compílolo, pruébelo y estudie su comentario.

```

1 importe java.util.Scanner;
2
3 //Buscar un carácter concreto dentro de una cadena de texto cualquiera.
4 public class BúsquedaCaracter {
5
6     public static void main(String[] args) {
7
8         Scanner lector = new Scanner(System.in);
9
10        System.out.println("Escribe una línea de texto cualquiera y pulsa retorno:");
11        String text = lector.nextLine();
12
13        System.out.println("¿Qué carácter quieres buscar? ");
14        String charText = lector.next();
15        lector.nextLine();
16
17        char charBuscar = charText.charAt(0);
18
19        int posInit = text.indexOf(charCerca);
20        int posFi = text.lastIndexOf(charCerca);
21        if (posInit > -1){
22            System.out.println("Las apariciones del carácter " + charCerca + " son:");
23            System.out.println("Primera vez - " + posInit);
24            System.out.println("Último golpe - " + posFi);
25
26            System.out.println("Este carácter no se encuentra en el texto.");
27        }
28
29    }
30
31 }

```

Reto 3: haga un programa que, en una serie de cadenas de texto pasadas como argumentos en el método principal, cuente cuántas contienen el carácter 'a'.

2.3.2 Comparación

Una tarea que se realiza muy a menudo con cadenas de texto es ver si un texto entrado por el usuario corresponde a una opción concreta entre las disponibles. Esto es especialmente frecuente en el caso de entrada de datos vía argumentos del método principal, ya que en muchos programas estos argumentos indican opciones para la ejecución.

Desgraciadamente, los objetos no aceptan ningún tipo de operación entre ellos, y eso incluye aquellas vinculadas a los operadores relacionales. Por tanto, no se pueden comparar directamente tampoco, tal y como se haría entre valores de tipos primitivos. También es necesario usar algún método disponible en la clase a la que pertenecen. Si la su clase no ofrece ningún método que le ayude a comparar, entonces es imposible compararlos.

La clase String ofrece distintos métodos para comparar cadenas de texto.

- `equalsIgnoreCase(textoPerComparar)` evalúa un valor booleano (`true/false`) según si la cadena de texto contenida en la variable en la que se invoca y el texto usado como a parámetro son idénticos o no. El texto usado como parámetro puede ser tanto uno literal, "...", como otra variable en la que haya almacenada una otra cadena de texto.

`equalsIgnoreCase(texto)`

Este otro método hace la misma función, pero ignorante mayúsculas y minúsculas.

Como ejemplo, se puede comprobar si los argumentos pasados en un programa corresponden a alguna de las opciones posibles. En el siguiente programa, es posible detectar si los argumentos corresponden a alguno de los aceptados: `-version`, `-help` y `-server`. Para cada caso, se hace una opción distinta. Si hay algún argumento que no es ninguno de estos tres, el programa informa que hay un error en los argumentos. Pruebe que es así.

```
1 //Comprueba si los argumentos se corresponden con un conjunto preestablecido.
2 public class CompruebaArgumentos {
3     public static void main (String args[]) {
4         //Nunca olvide comprobar si el array contiene argumentos!
5         if (args.length > 0) {
6             //Se va recorriendo el array y se mira qué valor hay.
7             for (int i = 0; i < args.length; i++) {
8                 if (args[i].equals("-version")) {
9                     System.out.println("Se ha usado el argumento \"-version\"");
10                } else if (args[i].equals("-help")) {
11                    System.out.println("Se utilizó el argumento \"-help\"");
12                } else if (args[i].equals("-server")) {
13                    System.out.println("Se utilizó el argumento \"-server\"");
14                } else {
15                    System.out.println("El argumento \"" + args[i] + "\" no es válido!");
16                }
17            }
18        } else {
19            System.out.println("No había ningún argumento.");
20        }
21    }
22 }
```

En ocasiones puede ser útil comparar dos cadenas de texto desde el punto de vista de el orden alfabético, es decir, de cuál sería su posición relativa en un diccionario.

Este tipo de comparación sería equivalente a los operadores relacionales “mayor que” (>) y “menor que” (<).

`compareToIgnoreCase(texto)`

Este método es el equivalente ignorando mayúsculas y minúsculas.

- `compareTo(textoPerComparar)` se comporta de forma similar a `compareToIgnoreCase`, pero en este caso evalúa a 0, cualquier valor positivo o cualquier valor negativo dependiendo de si la cadena almacenada en la variable en la que se invoca el método es alfabéticamente igual, mayor o menor (respectivamente) que la usada como parámetro.

La mejor forma de verlo es con un ejemplo. En el siguiente programa es necesario adivinar un texto secreto partiendo del hecho de que, para cada respuesta, el programa le dice como pista cuál es la posición relativa de la palabra secreta según su orden alfabético. Reflexione sobre qué diferencias hay entre este código, que trata cadenas de texto, y un programa que hiciera exactamente lo mismo con valores de tipo entero (en este caso, por orden numérico).

```
1 import java.util.Scanner;
2 //Juego de adivinar una palabra, dando pistas de su orden alfabético.
3 public class AdivinaPalabra {
4     //La palabra para adivinar es "objeto".
5     public static final String PALABRA_SECRETA = "java";
6     public static void main (String[] args) {
7         Scanner lector = new Scanner(System.in);
8         System.out.println("Empezamos el juego.");
9         System.out.println("Adivina el valor de la palabra del diccionario.");
10        boolean haEncertado = false;
11        while (!haEncertado) {
12            System.out.print("¿Qué palabra crees que es? ");
13            String palabraUsuario = lector.next();
14            lector.nextLine();
15            int posicio = palabraUsuario.compareTo(PALABRA_SECRETA);
16            if (posición < 0) {
17                //La palabra del usuario es anterior a la secreta.
18                System.out.println("¡Has fallado! La palabra va después...");
19            } else if (posición > 0) {
20                //La palabra del usuario es posterior a la secreta
21                System.out.println("¡Has fallado! La palabra va antes...");
22            } else {
23                //Si vale 0, es que se ha acertado.
24                haEncertado = true;
25            }
26        }
27        System.out.println("Enhorabuena. ¡Has acertado!");
28    }
29 }
```

Reto 4: usando el método `compareTo`, ordene un array de cadenas de texto. Por facilitar la entrada de los datos, este array puede estar determinado directamente a partir de argumentos en la ejecución del programa (la variable `args`).

2.3.3 Creación de subcadenas

El método `charAt` permite la extracción de datos de una cadena de texto, pero sólo de caracteres individuales. Sin embargo, la aplicabilidad de este mecanismo es limitada.

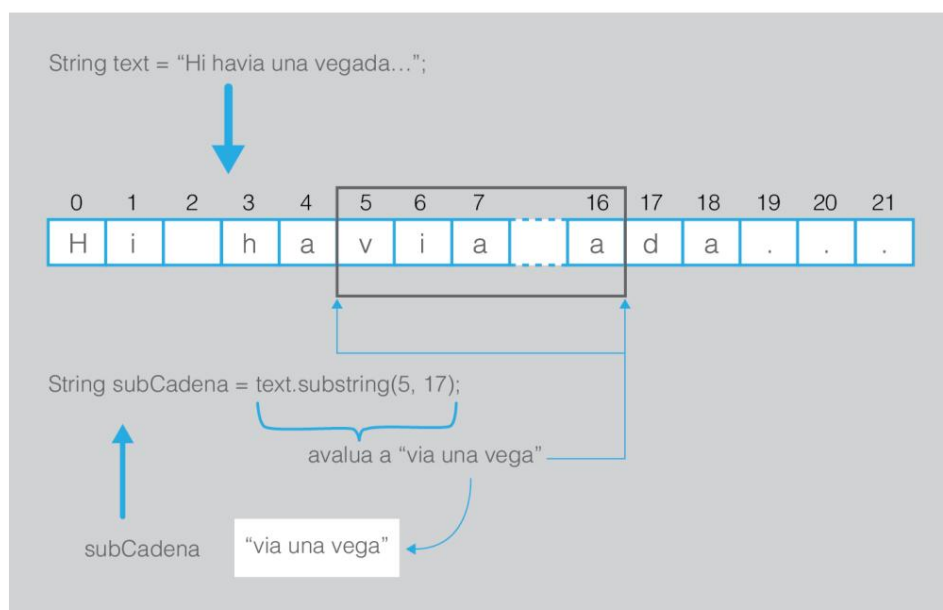
Muy a menudo se encontrará que, en realidad, le resultaría más cómodo poder extraer cadenas de texto completas contenidas dentro de otra cadena. Por ejemplo, partes de una frase o palabras individuales. La clase String ofrece dos métodos que permiten hacer esto.

- `substring(posiciónInicial, posiciónFinal)` evalúa una nueva cadena de texto compuesta por los caracteres que van desde `posiciónInicial` hasta `posiciónFinal - 1`.

Debe tener cuidado al usar este método, ya que los valores deben estar dentro del rango permitido. Si alguno supera el valor de las posiciones que ocupan los caracteres dentro de la cadena de texto, se producirá un error.

La figura 2.7 muestra un esquema del comportamiento del método `substring` si se extrae una subcadena de la frase “Érase una vez...”.

Figura 2. 7. Comportamiento del método “substring”



Como ejemplo del funcionamiento de la invocación de este método, a continuación encontrará el código de un programa que extrae la primera y la última palabra de una frase y las muestra por pantalla. Esto es equivalente a mostrar una subcadena de texto entre el primer y último espacio en blanco. Si hay menos de tres palabras, entonces no muestra nada.

```
1 importe java.util.Scanner;
2 //Un programa que extrae todo el texto de una frase, excepto la primera y la
  última palabra.
3 public class ExtraerPalabras {
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         System.out.println("Escribe una frase de texto y pulsa retorno.");
7         String text = lector.nextLine();
8         //Buscar el primer y último espacio en blanco.
9         int inicioSubcadena = text.indexOf(' ');
10        int fiSubcadena = text.lastIndexOf(' ');
11        System.out.println("El texto sin la primera y la última palabra es:");
12        if (inicioSubcadena == fiSubcadena) {
13            //O bien no hay espacios (ambos métodos evalúan a -1).
```

```
14      //O bien sólo hay dos palabras (ambos métodos evalúan la misma
      posición).
15      //No se muestra nada.
16      System.out.println("(No hay nada que escribir...)");
17  } else {
18      //Se muestra la cadena intermedia.
19      //La segunda palabra comienza una posición después del primer espacio en blanco
      .
20      //La última palabra comienza una posición después del último espacio en
      blanco.
21      String textFinal = text.substring(inicioSubcadena + 1, fiSubcadena);
22      System.out.println(textFinal);
23  }
24  }
25 }
```

Reto 5: haga un programa que muestre por pantalla cada una de las palabras individuales de una frase en líneas distintas. Para ello, antes tendrá que ir averiguando las posiciones donde hay espacios en blanco para poder delimitar dónde comienza y termina cada palabra.

Para obtener las palabras de una frase, no es necesario buscar dónde hay espacios e ir extrayendo el texto entre éstos. Existe otro método que facilita mucho más el trabajo.

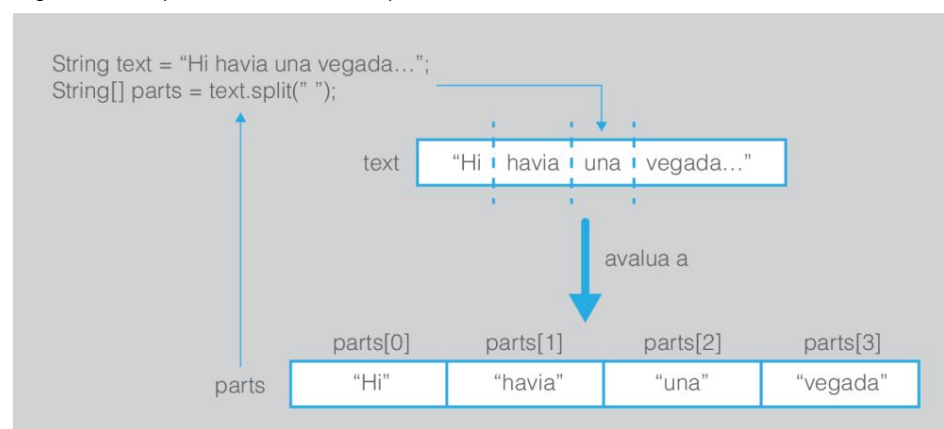
- `split(textSeparación)` evalúa un array de cadenas de texto, en el que cada posición está el resultado de hacer lo siguiente. Se toma la cadena de texto original y se separa en trozos usando como divisor `textoSeparación`. Cada trozo, una nueva cadena de texto, se ubica en una posición del array. El valor de `textoSeparación` debe ser una cadena de texto.

Por ejemplo, si como texto de separación de este método se usa un espacio en blanco, " ", una frase se dividiría en palabras. El resultado sería un array de cadenas de texto en las que en cada posición hay cada una de las palabras. Ahora bien, el separador puede ser cualquier texto que desee (comas, puntos, combinaciones de letras, etc.).

La figura 2.8 muestra un esquema del comportamiento de este método si se divide la frase "Había una vez—" usando como texto de separación el espacio en blanco, por lo que queda dividida en palabras.

Recuerde que el contenido de la cadena original nunca se modifica.

Figura 2. 8. Comportamiento del método "split"



Como ejemplo, estudie el código fuente siguiente, que pertenece a un programa para invertir el orden de las palabras de una frase. Pruebe en su entorno de trabajo que es así.

```
1 importe java.util.Scanner;
2 //Programa que invierte el orden de las palabras de una frase.
3 public class InvierteOrdenPalabras {
4     public static void main(String[] args) {
5         Scanner lector = new Scanner(System.in);
6         System.out.println("Escribe una línea de texto cualquiera y pulsa retorno:");
7         String text = lector.nextLine();
8         //Dividimos la frase en palabras, separadas por espacios.
9         String[] palabras = text.split(" ");
10        System.out.println("El texto invertido es:");
11        //Imprimimos las palabras en orden inverso.
12        for (int i = palabras.length - 1; i >= 0 ; i--) {
13            System.out.print(palabras[i]);
14            //El método split elimina el texto de separación (en este caso, los espacios
15            )
16            System.out.print(" ");
17        }
18        System.out.println();
19 }
```

Reto 6: realice un programa que genere automáticamente acrónimos. Partiendo de una frase determinada, debe mostrar por pantalla el texto compuesto por las iniciales de cada palabra individual que forma la frase.

2.3.4 Transformaciones entre cadenas de texto y tipos primitivos

En muchos casos, una parte importante de los programas se basa en las operaciones realizadas sobre datos de tipos primitivos. Ahora bien, de cara a la interacción con el usuario, ya sea tanto para mostrarlas como para leerlas, es necesario operar con cadenas de texto. Por tanto, un mecanismo útil es el de la transformación de tipos primitivos en cadenas de texto y viceversa.

Conversión de tipo primitivo a cadena de texto

El caso más sencillo es la transformación de un dato de tipo primitivo a un objeto de la clase String. Si los valores que se quieren representar forman parte de una frase o una cadena de texto más larga (por ejemplo: "El resultado es ", y el valor), tan sólo es necesario usar la operación suma, como se ha explicado en el apartado 1.1.5. Java se encarga de realizar la conversión de tipos automáticamente si en alguno de los operandos dentro de una concatenación existe algún dato de tipo primitivo. Para el caso de mostrar directamente un valor por pantalla, la instrucción `System.out.println` también transforma directamente cualquier tipo primitivo que se le proporcione en una cadena de texto que será mostrada por pantalla.

Si sólo queremos convertir un valor directamente, sin que éste tenga que estar incluido dentro de una frase o con el único objetivo de mostrarlo por pantalla posteriormente, entonces se puede usar la instrucción `String.valueOf(valor)`. Donde pone valor se puede usar cualquier dato correspondiente a cualquier tipo primitivo, ya sea una

variable, un literal o expresión. Esta instrucción evalúa su equivalente a cadena de texto.

Dadas las capacidades innatas de la concatenación y la impresión por pantalla para transformar datos de tipos primitivos en cadenas de texto automáticamente, los casos en los que resulta aplicable el uso de la instrucción `String.valueOf(valor)` es limitado. En la inmensa mayoría de los casos, si desea convertir un tipo primitivo en cadena de texto será para mostrarlo por pantalla, ya sea solo o dentro de una frase. Sin embargo, puede haber casos en los que tratar un dato como cadena de texto en lugar de como valor numérico puede facilitar la tarea del programador. Por ejemplo, cuando se desea tratar este valor como una secuencia de dígitos individuales.

Suponga un programa que muestra los dígitos en orden inverso de un número real de cierta longitud, con cualquier número de decimales. Piense un poco y verá que, tratándolo como valor numérico y usando operaciones entre reales, no es un problema cuya resolución resulte evidente. Éste es un caso en el que el algoritmo para resolver el problema es algo más sencillo si el valor a procesar se trata como una secuencia de caracteres que si se trata como un número. Sólo habría que realizar un recorrido por cada uno de los caracteres individuales (las cifras) usando una estructura de repetición. Por tanto, es útil transformarlo en una cadena de texto y procesarlo como tal.

Pruebe en su entorno el código fuente, mostrado a continuación:

```
1 //Muestra las cifras de un número real en orden inverso. 2 public class InvertirOrden { public static void main
(String[] args) { float numero = 3.1415926535f; //Lo convertimos
3     en una cadena de texto.
4
5
6     String numeroText = String.valueOf(numero); //Recorremos
7     sus dígitos uno por uno, empezando por el final. for (int i = numeroText.length()
8         - 1; i >= 0; i--) {
9         //Imprimimos cada dígito individual (incluido el punto decimal).
10        System.out.print(numeroText.charAt(i));
11    }
12 }
13 }
```

Conversión de cadena de texto a tipo primitivo

Aunque ya sabe cómo leer datos de los tipos primitivos directamente desde el teclado, a veces, partiendo de una cadena de texto, le interesaría transformarla a un tipo primitivo (normalmente, un valor numérico). Por ejemplo, éste puede ser el caso de un programa no interactivo que obtiene los datos de entrada para el paso de argumentos al método principal. En este caso, siempre se parte de valores en formato cadena de texto y, por tanto, hay que realizar una conversión si alguno de los argumentos es un valor numérico con el que operar.

Para este caso, Java dispone de una serie de instrucciones, una para cada tipo primitivo, que sirve para realizar esta conversión. Ahora bien, como ocurre con las instrucciones de lectura de tipos primitivos por teclado, es necesario garantizar que la cadena de texto para convertir es correcta, ya que de lo contrario el programa dará un error.

Por ejemplo, no es posible convertir a un valor de tipo entero las cadenas de texto

"2,67334", "1a88", "true", etc. Estas instrucciones se enumeran en la tabla 2.1.

Donde dice texto es necesario poner un objeto de tipo String, ya sea mediante una variable, un literal o una concatenación de cadenas.

Tabla 2. 1. Instrucciones para convertir cadenas de texto en tipos primitivos

Instrucción	Tipo de dato convertido
Byte.parseByte(texto)	byte
Integer.parseInt(texto)	int
Double.parseDouble(texto)	double
Float.parseFloat(texto)	float
Long.parseLong(texto)	long
Short.parseShort(texto)	short

Por ejemplo, suponga un programa que escribe una cadena de texto un cierto número a veces. Para ello, se basa en dos argumentos en el método principal. El primero es la cadena de texto para escribir, y el segundo el número de veces (un valor entero). Observe su código, pruébelo y ejecútelo en su entorno. Observe también qué ocurre si el segundo argumento no es un valor de tipo entero.

```
1 //Muestra una cadena de texto N veces, partiendo de argumentos de entrada.
2 public class MuestraNVueltas {
3     public static void main (String[] args) {
4         //Se mira si existen los 2 argumentos que corresponden.
5         if (args.length != 2) {
6             System.out.println("El número de argumentos no es correcto!");
7         } else {
8             //Hay 2 argumentos, el segundo es el número de veces.
9             int numVegas = Integer.parseInt(args[1]);
10            //Hacemos una estructura de repetición. Es necesario mostrar el primer argumento N
              veces.
11            for (int i = 0 ; i < numVegas; i++) {
12                System.out.println(args[0]);
13            }
14        }
15    }
16 }
```

Reto 7: haga un programa que calcule la división entre dos valores reales, haciendo que estos valores sean entrados como argumentos en el método principal.

2.4 Soluciones de los retos propuestos

Reto 1:

```
1 public class InvierteCadena {
2     public static void main (String[] args) {
3         String holaMon = "¡Hola, mundo!";
4         //Se recorren las posiciones de la cadena de texto una por una.
5         for (int i = holaMon.length() - 1; i >= 0; i--) {
6             System.out.print(holaMon.charAt(i));
7         }
8         System.out.println();
9     }
10 }
```

Reto 2:

```
1 importe java.util.Scanner;
2 public class LecturaCaracterTresIntentos {
3     //El número de intentos se declara como una constante.
4     public static final int INTENTES = 3;
5     public static final char RESPUESTA_CORRECTA = 'b';
6     public static void main (String[] args) {
7         Scanner lector = new Scanner(System.in);
8         System.out.println("Adivina la pregunta.");
9         System.out.println("¿Cuál de los siguientes no es un tipo primitivo?");
10        System.out.println("a) Enter");
11        System.out.println("b) Scanner");
12        System.out.println("c) Carácter");
13        System.out.println("d) Booleano");
14        //Es necesario un contador de intentos.
15        int intentos = 0;
16        //Hay que saber si se ha acertado (semáforo).
17        boolean acertado = false;
18        //Se pregunta mientras no se acierte ni se agoten los intentos.
19        while ((intentos < INTENTOS)&&(!acertado)) {
20            System.out.print("Tu respuesta es la opción: ");
21            //Se lee la cadena de texto.
22            String palabra = lector.next();
23            //¿Es una palabra de un único carácter?
24            if (palabra.length() == 1) {
25                //Se extrae el carácter de la cadena de texto.
26                char caracter = palabra.charAt(0);
27                //¿Es un carácter válido? (a, b, cod).
28                if ((caracter >= 'a')&&(caracter <= 'd')) {
29                    //¿La respuesta final es correcta?
30                    if (caracter == RESPUESTA_CORRECTA) {
31                        System.out.println("Efectivamente, la respuesta era " +
32                            RESPUESTA_CORRECTA + ".");
33                        acertado = true;
34                    } else {
35                        System.out.println("La respuesta " + caracter +
36                            " es incorrecta.");
37                        intentos++;
38                    }
39                } else {
40                    System.out.println("Si existe este " + caracter + " es una opción incorrecta.");
41                    error, no se agotará ningún intento...
42                }
43            } else {
44                //No lo era.
45                System.out.println("La palabra " + palabra + " no es un carácter individual.");
46                error, no se agotará ningún intento...
47            }
48        }
49    }
```

```
46     lector.nextLine();
47     }
48     if (intentos >= INTENTOS) {
49         System.out.println("Has agotado todos tus intentos.");
50     }
51 }
52 }
```

Reto 3:

```
1 public class CuentaArgumentosA {
2     public static void main(String[] args) {
3         //Un contador de palabras con 'a'.
4         int numAs = 0;
5         //Hay que recorrer el array de argumentos.
6         for (int i = 0; i < args.length; i++) {
7             //Se mira si hay alguna letra 'a' en la cadena de texto.
8             if (-1 != args[i].indexOf("a")) {
9                 numAs++;
10            }
11        }
12        System.out.println("El número de argumentos con el carácter 'a' es " + numAs)
13        ;
14    }
15 }
```

Reto 4:

```
1 public class OrdenaArgumentos {
2     public static void main (String[] args) {
3         for (int i = 0; i < args.length - 1; i++) {
4             for(int j = i + 1; j < args.length; j++) {
5                 if (args[i].compareTo(args[j]) > 0) {
6                     String cambio = args[i];
7                     args[i] = args[j];
8                     args[j] = cambio;
9                 }
10            }
11        }
12        //Lo mostramos por pantalla.
13        System.out.println("Las cadenas ordenadas son:");
14        for (int i = 0; i < args.length;i++) {
15            System.out.println(args[i]);
16        }
17    }
18 }
```

Reto 5:

```
1 importe java.util.Scanner;
2 public class ObtenerPalabras {
3     public static void main(String[] args) {
4         Scanner lector = new Scanner(System.in);
5         System.out.println("Escribe una frase de texto y pulsa retorno:");
6         String text = lector.nextLine();
7         boolean fin = false;
8         //Mediante un bucle, se irá acortando el texto palabra por palabra.
9         do {
10            //La primera palabra va del inicio al primer espacio.
11            int primeroEspacio = text.indexOf(" ");
12            //No hay espacios. No hay ninguna palabra más que escribir.
13            if (primeroEspacio == -1) {
14                System.out.println(texto);
15                fin = true;
16            } else {
17                //Hay un espacio. Extraer palabra y mostrarla.
18                String palabra = text.substring(0, primeroEspacio);
19                System.out.println(palabra);
20                //Se elimina la palabra del texto, por lo que se acorta.
21                texto = text.substring(primeroEspacio + 1, text.length());
22            }
23        } while (!fin);
24    }
25 }
```

Reto 6:

```
1 importe java.util.Scanner;
2 public class GeneradorAcronimos {
3     public static void main(String[] args) {
4         Scanner lector = new Scanner(System.in);
5         System.out.println("Escribe una línea de texto cualquiera y pulsa retorno:");
6         String text = lector.nextLine();
7         //Dividimos la frase en palabras, separadas por espacios.
8         String[] palabras = text.split(" ");
9         //Recorremos las palabras y escribimos cada inicial.
10        for(int i = 0; i < palabras.length; i++) {
11            System.out.print(palabras[i].charAt(0));
12        }
13        System.out.println();
14    }
15 }
```

Reto 7:

```
1 public class DividirArgumentosReales {
2     public static void main(String[] args) {
3         //Está bien comprobar si el número de argumentos es correcto.
4         if (args.length != 2) {
5             System.out.println("El número de argumentos es incorrecto.");
6         } else {
7             //Conversión de tipos.
8             double dividendo = Double.parseDouble(args[0]);
9             double divisor = Double.parseDouble(args[1]);
10            double resultado = dividendo/divisor;
11            System.out.println("El resultado es" + resultado);
12        }
13    }
14 }
```